



第七节：特征检测与匹配

杨聪

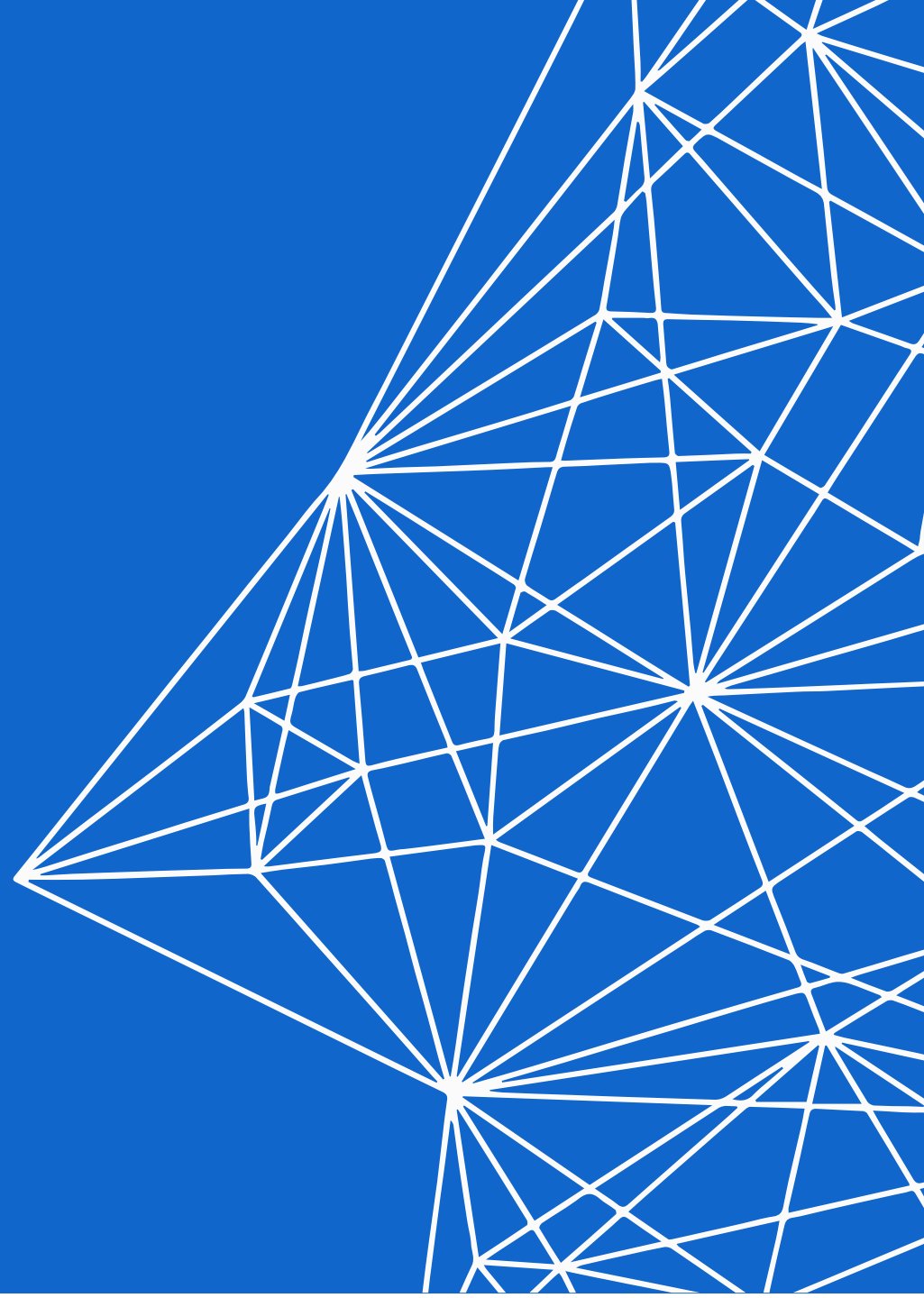




目录

CONTENTS

1. 图像边缘检测
2. 直线检测
3. 角点检测
4. 图像局部特征点检测与描述
5. 消失点检测
6. 特征点匹配



为什么要进行边缘检测

边缘检测一般是识别目标图像中亮度变化明显的像素点，因为显著变化的像素点通常反映了图像变化比较重要的地方。



边缘检测算法，其目标是找到一个最优的边缘，其最优边缘的定义是：

- 1.好的检测 --算法能够尽可能多地标示出图像中的实际边缘
- 2.好的定位 --标识出的边缘要与实际图像中的实际边缘尽可能接近
- 3.最小响应 --图像中的边缘只能标识一次，并且可能存在的图像噪声不应该标识为边缘

Canny边缘检测理论

Canny 是一种常用的边缘检测算法. 其是在 1986 年 John F.Canny 提出的.

Canny 是一种 multi-stage 算法, 分别如下:

- [1] - 应用高斯滤波来平滑图像, 去除噪声
- [2] - 找寻图像的强度梯度(intensity gradients)
- [3] - 应用非极大值抑制 (non-maximum suppression) 技术来消除边缘误检 (本来不是但检测出来是)
- [4] - 应用双阈值的方法来决定可能的(潜在)边界
- [5] - 利用滞后技术来跟踪边界



John Canny

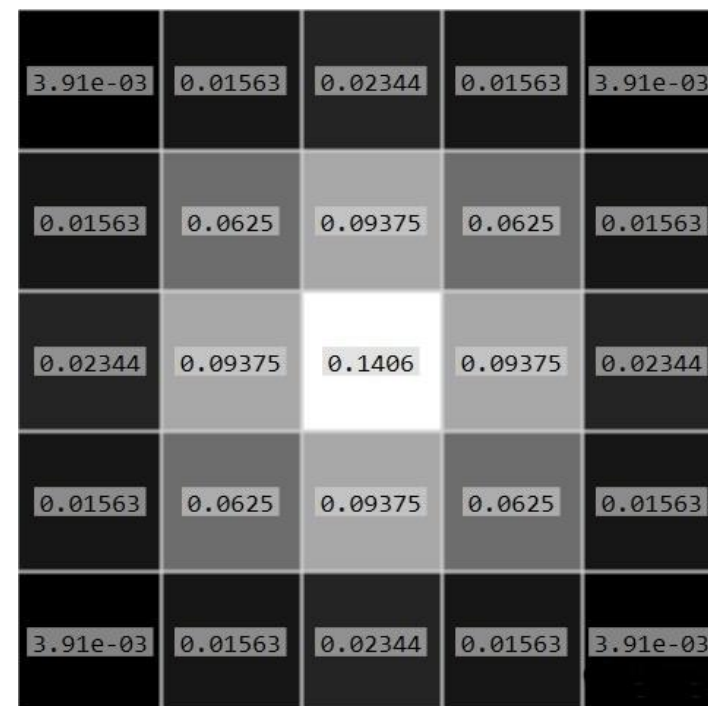
John Canny, University of California, Berkeley

Canny边缘检测理论

1. 应用高斯滤波来平滑图像，去除噪声

$$G(u, v) = \frac{1}{2\pi\sigma^2} e^{-(u^2+v^2)/(2\sigma^2)}$$

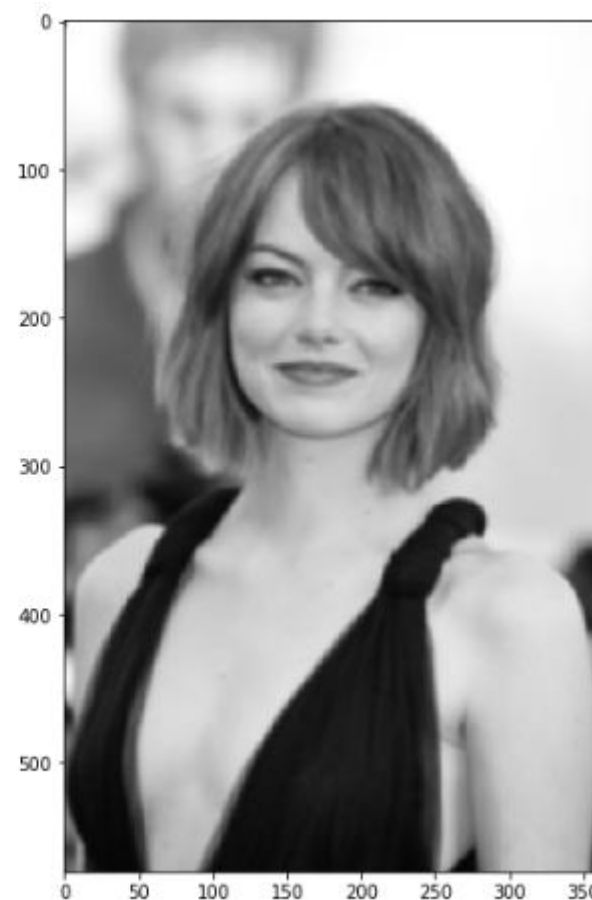
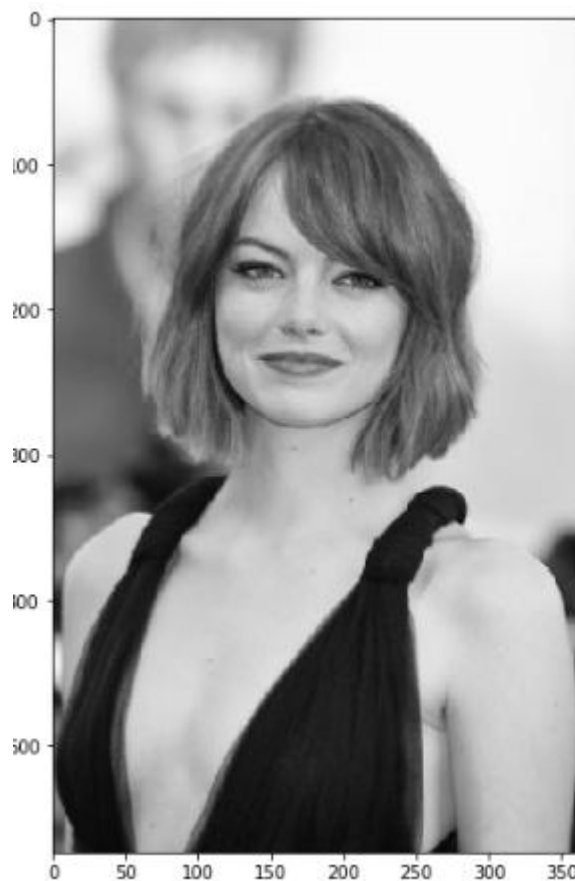
相邻像素随着距离原始像素越来越远，其权重也越来越小



半径为2，标准差为0时的二维高斯权重分布

Canny边缘检测理论

1. 应用高斯滤波来平滑图像，去除噪声



John Canny, University of California, Berkeley

Canny边缘检测理论

2. 找寻图像的强度梯度(intensity gradients)

计算图像梯度能够得到图像的边缘，因为梯度是灰度变化明显的地方，而边缘也是灰度变化明显的地方。使用**Sobel**或其它算子得到不同方向的梯度值 $g_x(m, n)$, $g_y(m, n)$ 。通过以下公式计算梯度值和梯度方向

$$G(m, n) = \sqrt{g_x(m, n)^2 + g_y(m, n)^2}$$

$$\theta = \arctan \frac{g_y(m, n)}{g_x(m, n)}$$

-1	-2	-1
0	0	0
1	2	1

水平

-1	0	1
-2	0	2
-1	0	1

垂直

0	1	2
-1	0	1
-2	-1	0

对角线

-2	-1	0
-1	0	1
0	1	2

Sobel算子:

John Canny, University of California, Berkeley

Canny边缘检测理论

2. 找寻图像的强度梯度(intensity gradients)

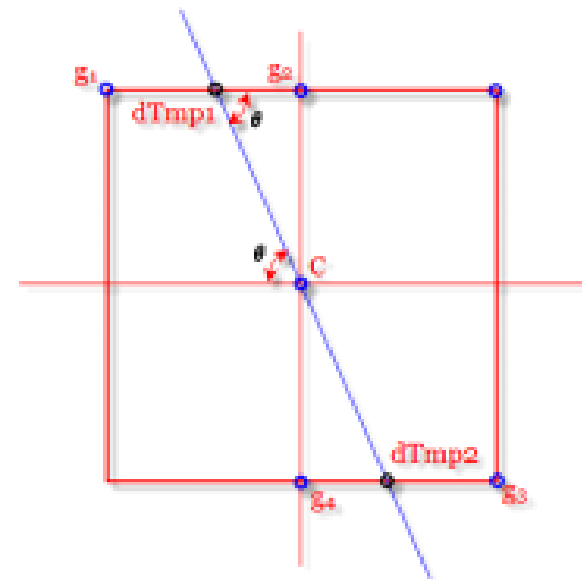


Canny边缘检测理论

3. 应用非极大值抑制 (Non-Maximum Suppression, NMS)

从上一步得到的梯度图像存在**边缘粗宽**、**弱边缘干扰**等众多问题，现在我们可以使用**非极大值抑制**来寻找像素点**局部最大值**，将非极大值所对应的灰度值置**0**，这样可以剔除一大部分非边缘的像素点。

如右图所示，C表示为当前非极大值抑制的点，g1-4为它的8连通邻域点，图中蓝色线段表示上一步计算得到的角度图像C点的值，即梯度方向，第一步先判断C灰度值在8值邻域内是否最大，如是，则继续检查图中梯度方向交点dTmp1,dTmp2值是否大于C，如C点大于dTmp1,dTmp2点的灰度值，则认定C点为极大值点，置为1，因此最后生成的图像应为一副**二值图像**，边缘理想状态下都为**单像素边缘**

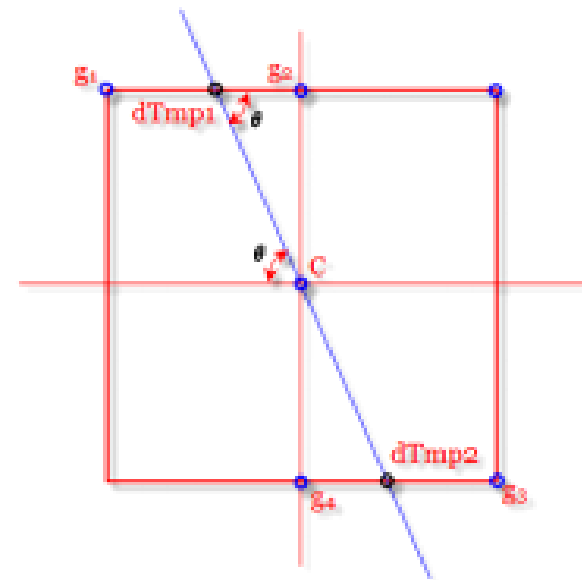


Canny边缘检测理论

3. 应用非极大值抑制 (Non-Maximum Suppression, NMS)

从上一步得到的梯度图像存在边缘粗宽、弱边缘干扰等众多问题，现在我们可以使用**非极大值抑制**来寻找像素点**局部最大值**，将非极大值所对应的灰度值置**0**，这样可以剔除一大部分非边缘的像素点。

如右图所示，C表示为当前非极大值抑制的点，g1-4为它的8连通邻域点，图中蓝色线段表示上一步计算得到的角度图像C点的值，即梯度方向，第一步先判断C灰度值在8值邻域内是否最大，如是则继续检查图中梯度方向交点dTmp1,dTmp2值是否大于C，如C点大于dTmp1,dTmp2点的灰度值，则认定C点为极大值点，置为1，因此最后生成的图像应为一副**二值图像**，边缘理想状态下都为**单像素边缘**



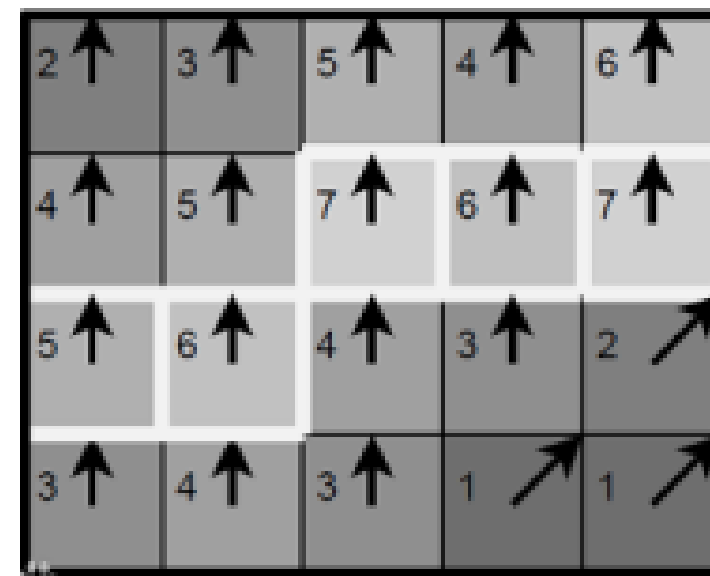
(其中需要注意的是梯度方向交点并不一定落在8邻域所在8个点的位置，因此dTmp1和dTmp2实际应用中是使用相邻两个点的双线性插值所形成的灰度值)

Canny边缘检测理论

3. 应用非极大值抑制 (Non-Maximum Suppression, NMS)

从上一步得到的梯度图像存在边缘粗宽、弱边缘干扰等众多问题，现在我们可以使用**非极大值抑制**来寻找像素点**局部最大值**，将非极大值所对应的灰度值置**0**，这样可以剔除一大部分非边缘的像素点。

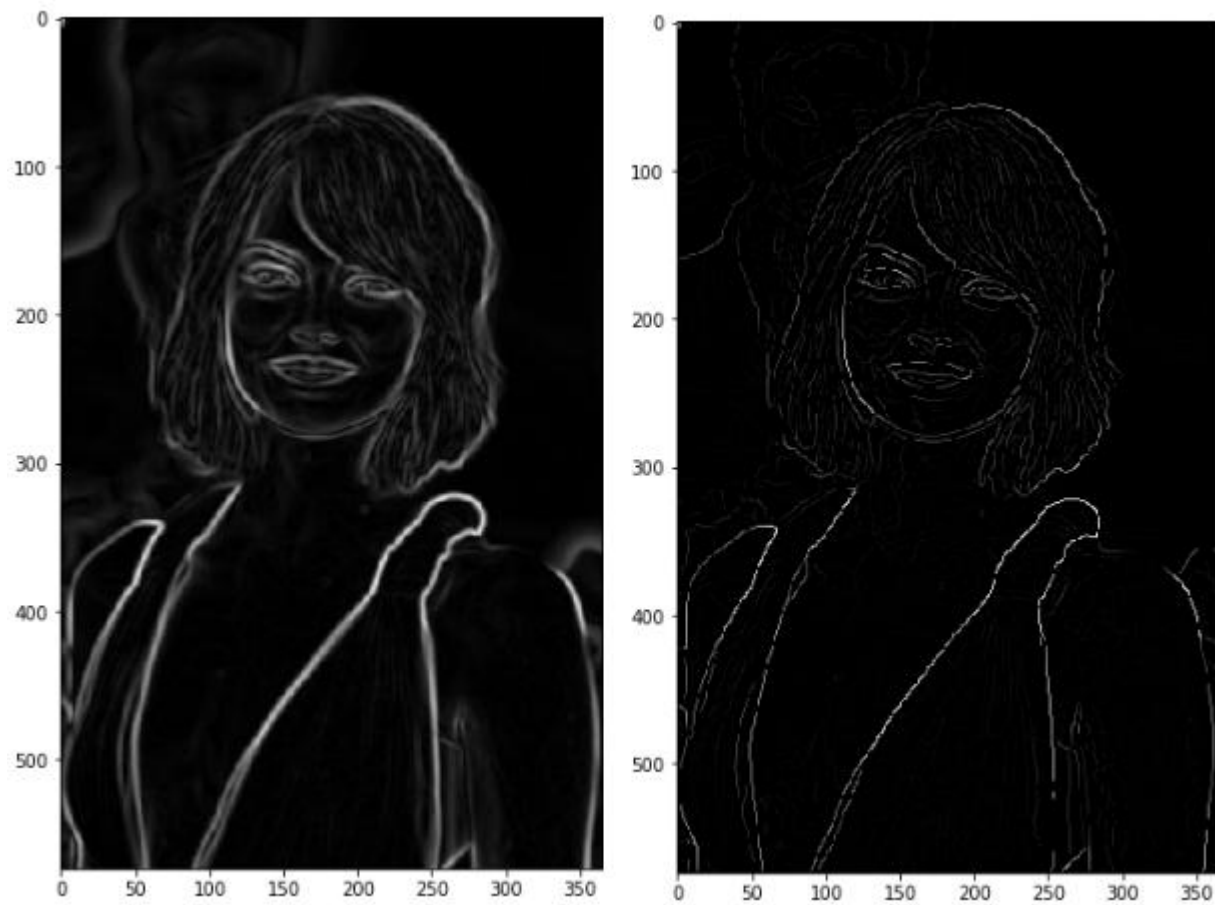
如图所示，其中梯度方向均为垂直向上，经过非极大值抑制后取梯度方向上最大值为边缘点，形成细且准确的单像素边缘



John Canny, University of California, Berkeley

Canny边缘检测理论

3. 应用非极大值抑制 (Non-Maximum Suppression, NMS)



John Canny, University of California, Berkeley



Canny边缘检测理论

4. 应用双阈值的方法来决定可能的(潜在)边界

经过以上三步之后得到的边缘质量已经很高了，但还是存在很多**伪边缘**，因此Canny算法中所采用的算法为**双阈值法**，具体思路为选取两个阈值，将小于低阈值的点认为是假边缘置0，将大于高阈值的点认为是强边缘置1，介于中间的像素点需进行进一步的检查

根据高阈值图像中把边缘链接成轮廓，当到达轮廓的端点时，该算法会在断点的8邻域点中寻找满足低阈值的点，再根据此点收集新的边缘，直到整个图像闭合，

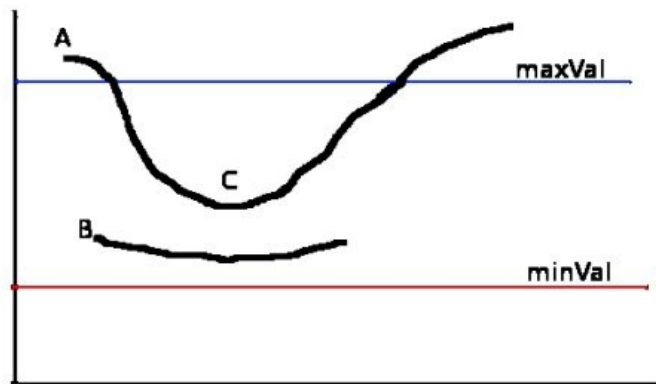
Canny边缘检测理论

4. 应用双阈值的方法来决定可能的(潜在)边界

需要设定两个阈值, minVal 和 maxVal .

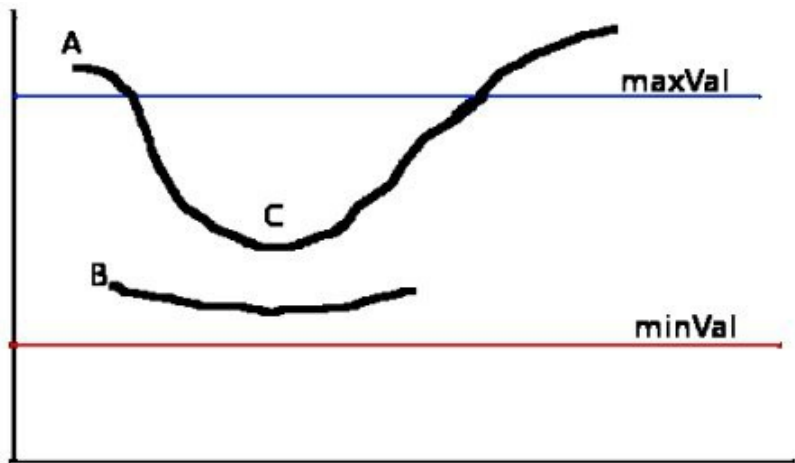
任何边缘的强度梯度大于 maxVal 的确定为边缘; 而小于 minVal 的确定为非边缘, 并丢弃.

位于 maxVal 和 minVal 阈值间的边缘为待分类边缘, 或非边缘, 基于连续性进行判断. 如果边缘像素连接着 "确定边缘(sure-edge)" 像素, 则认为该边缘属于真正边缘的一部分; 否则, 丢弃该边缘.



Canny边缘检测理论

4. 应用双阈值的方法来决定可能的(潜在)边界



边缘 A 大于 maxVal ，因此为“确定边缘(sure-edge)”。
虽然边缘 C 小于 maxVal ，但其连接着边缘 A，因此也认为是有效边缘，以得到完整的边缘曲线。

但，边缘 B 虽然大于 minVal ，并与边缘 C 位于相同的区域，但其没有与任何“确定边缘”相连接，因此，丢弃该边缘 B。

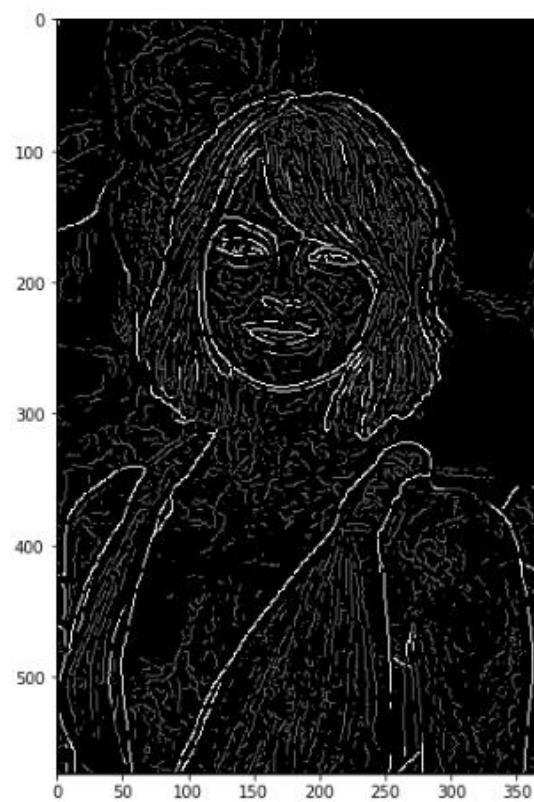
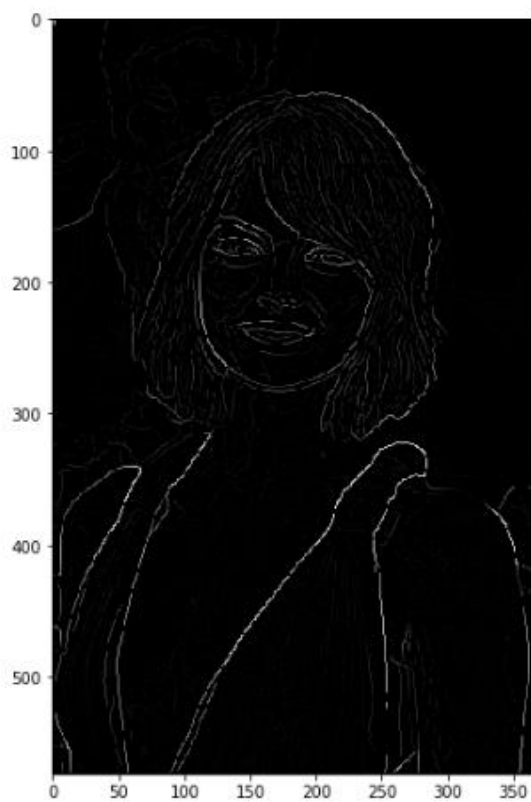
由上可见， minVal 和 maxVal 值的选择对于边缘检测的结果非常重要。

此外，该阶段的处理还移除小的像素噪声，因为假设边缘是长曲线。

最终，即可得到图片的有效边缘。

Canny边缘检测理论

4. 应用双阈值的方法来决定可能的(潜在)边界



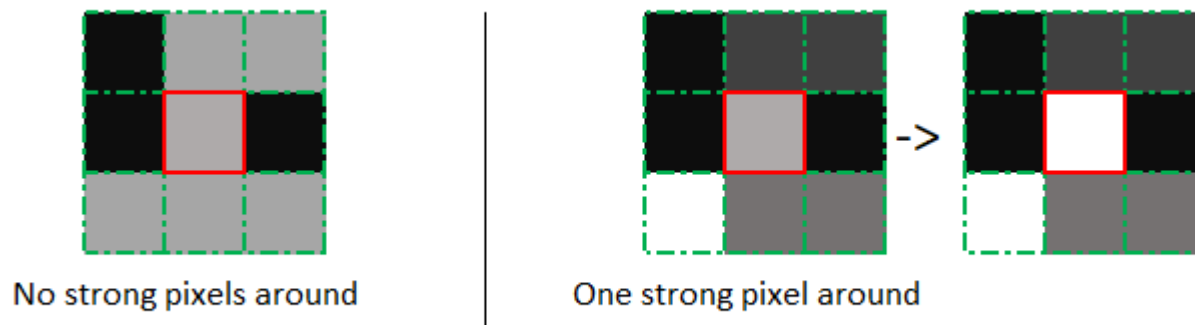
使用两个阈值比使用一个阈值更加灵活，但是它还是有阈值存在的共性问题。设置的阈值过高，可能会漏掉重要信息；阈值过低，将会把枝节信息看得很重要。很难给出一个适用于所有图像的通用阈值。目前还没有一个经过验证的实现方法。

看杨老师的例子： With max and min manual canny

weak pixels in gray and strong ones in white

Canny边缘检测理论

5. 利用滞后技术来跟踪边界

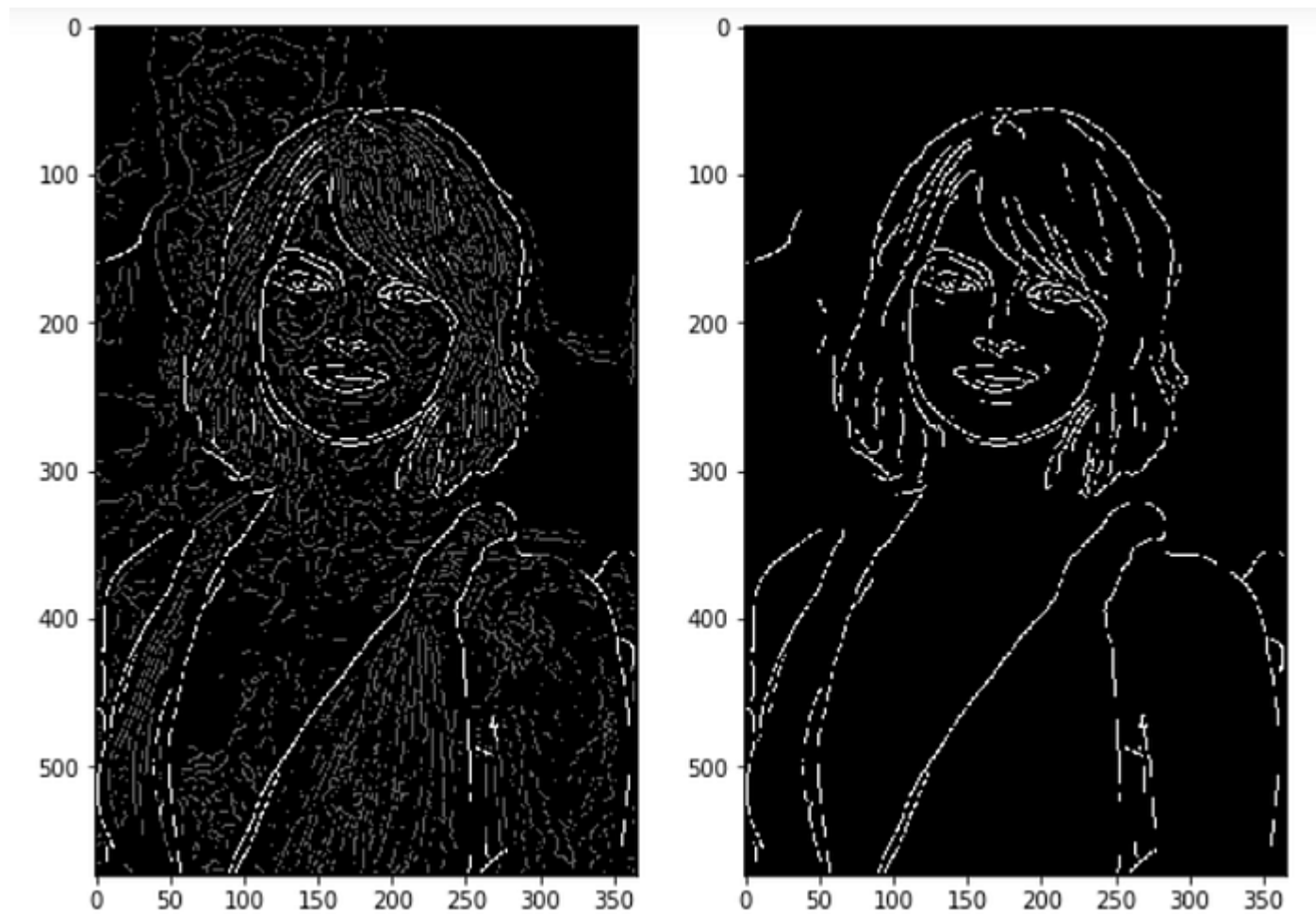


transforming weak pixels into strong ones, if and only if at least one of the pixels around the one being processed is a strong one

根据高阈值图像中把边缘链接成轮廓，当到达轮廓的端点时，该算法会在断点的8邻域点中寻找满足低阈值的点，再根据此点收集新的边缘，直到整个图像闭合

Canny边缘检测理论

5. 利用滞后技术来跟踪边界



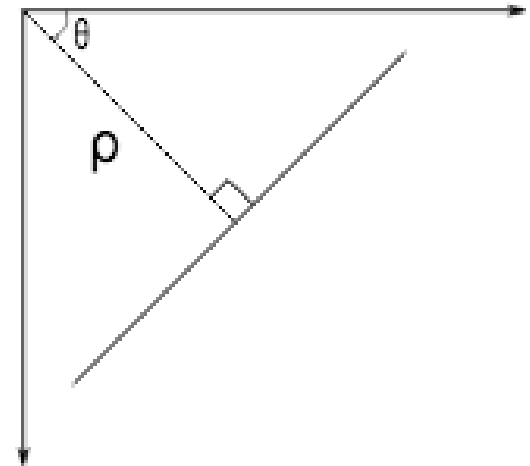
John Canny, University of California, Berkeley



霍夫变换 (Hough Transform)

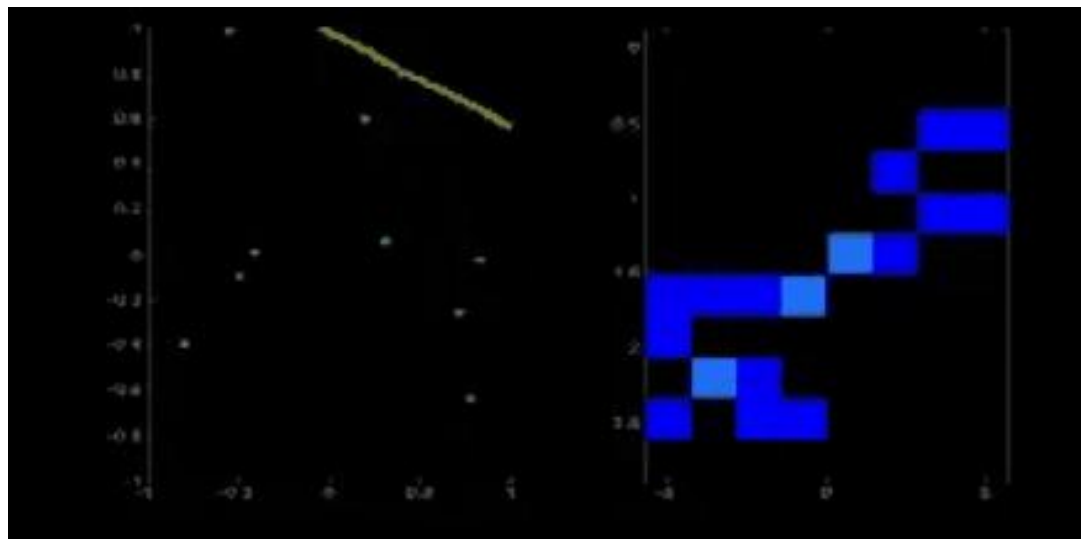
霍夫变换是图像处理必然接触到的一个算法，它通过一种投票算法检测具有特定形状的物体,该过程在一个参数空间中通过计算累计结果的局部最大值得到一个符合该特定形状的集合作为霍夫变换结果，该方法可以进行圆，直线，椭圆等形状的检测。在车道线检测中，当初考虑的一个方案便是采用霍夫变换检测直线进行车道线提取。

直线可以通过类似 $y = mx + c$ 或 极坐标 $\rho = x \cos\theta + y \sin\theta$ (其中 ρ 表示原点到直线的垂直距离， θ 是垂直线与水平轴之间的逆时针方向夹角，方向主要取决于如何放置坐标系)来表示。如右图：



直线检测

霍夫变换 (Hough Transform)

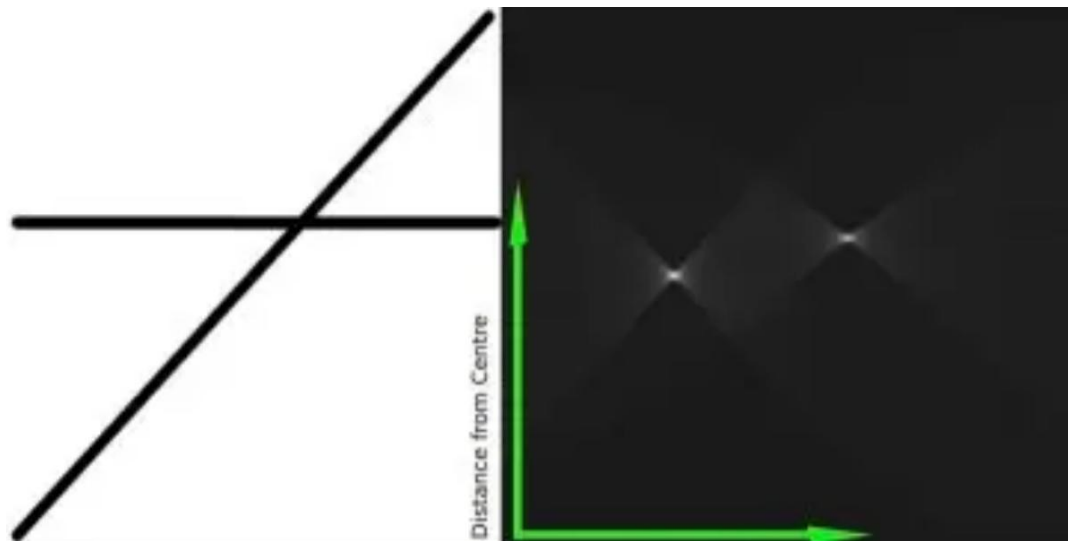


假设有一个100*100的图片，有一条水平线在中间，获取这条线的第一个点，已知它的(x, y)值，然后将 $\theta = 0, 1, 2, \dots, 180$ 放入线条方程中，获取 ρ 值。对于每个 (ρ, θ) 单元格，累加器中与之相关的 (ρ, θ) 单元格每次增加的值为1，所以在累加器中， $(50, 90) = 1$ 和一些其他单项相关联。然后取线条上的第二个点，进行以上相同的动作，增加与之相关联的 (ρ, θ) 单元格，此时 $(50, 90) = 2$ ，现在的操作都会增加 (ρ, θ) 的值。

继续为线条上的每个点进行相同的操作，在每个点， $(50, 90)$ 单元格可以增值或者计数，其他单元格可能不会被计数。通过这种方式，最后 $(50, 90)$ 单元格会得到最大的计数。如果在计数器中搜索最大计数，就可以得到 $(50, 90)$ 的值，表示在距离原点50， 90° 角的地方有一条线。

直线检测

霍夫变换 (Hough Transform)

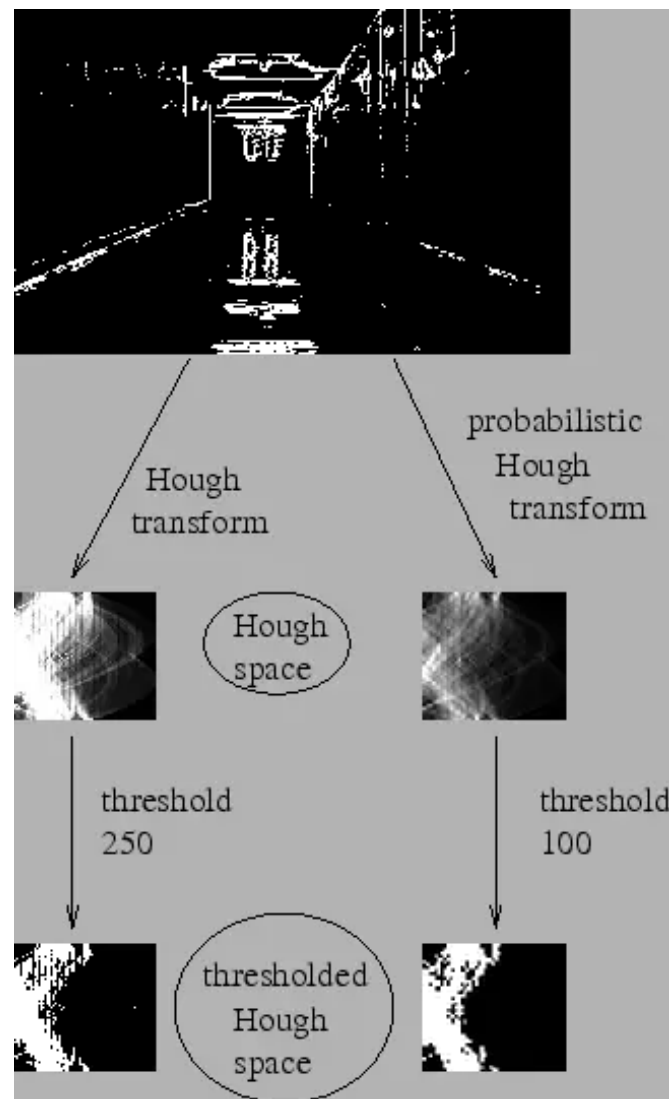


上图展示了累加模型，图中高亮的位置代表着图片中可能是线条的参数。

霍夫变换 (Hough Transform)

概率霍夫变换:

在霍夫变换中，可以通过2个变量来检测一条线上的点，但花费大量计算。概率霍夫变换是一个优化的霍夫变换，它不会计算所有的点，而是随机的选取一组足以识别直线的点，所以我们需要减少阈值。看下图关于霍夫变换和概率霍夫变换在霍夫空间的比较：





直线检测

霍夫变换 (Hough Transform)

霍夫变换: `cv2.HoughLines()`

它返回 (ρ, θ) 值的序列, ρ 单位像素, θ 单位弧度。第一个参数, 输入的图片是一个二进制图片, 在使用hough变换之前, 应用阈值或使用canny边缘检测。第二和第三个参数分别是 ρ 和 θ 的精度, 第4个参数是阈值, 指可以被认为是一个线条的最小计数值。由于计数值的多少取决于线上的点数, 所以这代表了可以被识别为线的最小长度。

概率霍夫变换: `cv2.HoughLinesP()`

函数有2个新的参数:

`minLineLength` - 线段的最小长度. Line segments shorter than this are rejected.

`maxLineGap` - 使程序识别线段为一条线的线段之间最大的空隙

看杨老师demo: `houghline.py`



直线检测

霍夫变换 (Hough Transform)

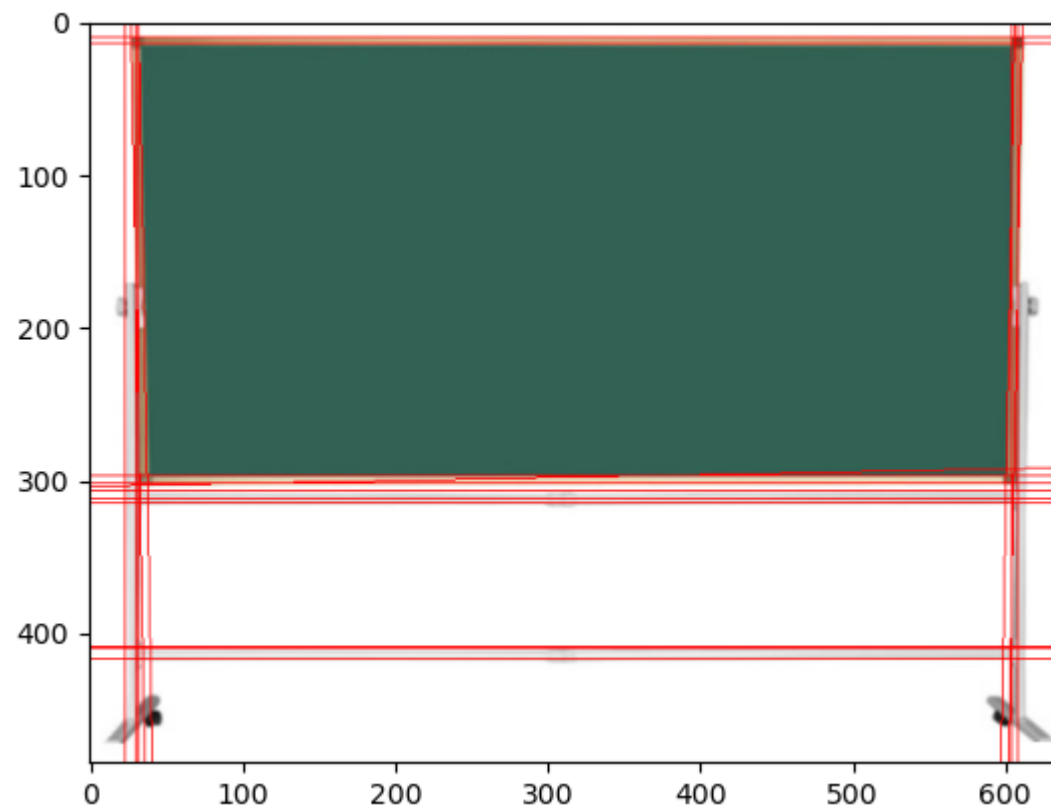
检测线: `cv2.HoughLinesP()`

检测圆: `cv2.HoughCircles()`

检测边缘: `cv2.findContours`

检测多边形: `cv2.approxPolyDP()`

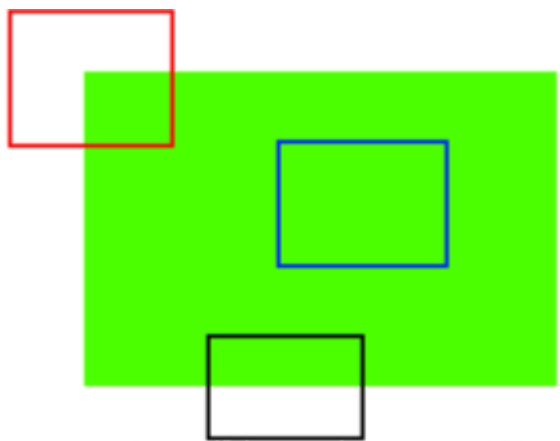
建议大家在网上找资料，动手实践一下



角点检测

Harris角点检测

简单粗暴的讲，**红色框框住的部分就是角点**，**黑色框框住的部分是边界**，**蓝色框框住的...就是普通的部分。**



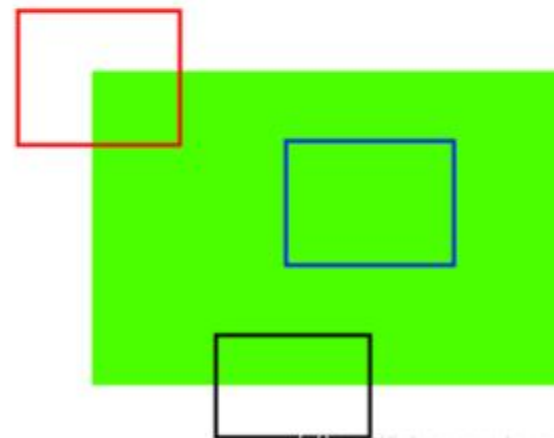
角点检测

Harris角点检测

再生动点:



A 和 B 是平而且它们的图像中很多地方存在。
C 和 D 是边界，可以大致找到位置。
E 和 F 是角点，可以迅速定位到。



https://blog.csdn.net/qj_40374812

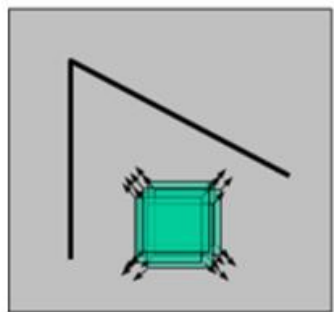
- A和B处于平坦区域，没有什么确切的特征，它们所在的位置有很多种可能；
- C和D要相对简单一些，它们是建筑物的边缘，我们可以找到一个大致的位置，但是要定位到精确的位置仍然很难。所以边缘是更好的特征，但还不够好。
- E和F是建筑的一些角落，可以很容易地发现它们的位置，因为对于建筑物角落这个图像片段，我们不管朝哪个方向移动，这个片段看起来都会不一样。

角点检测

Harris角点检测

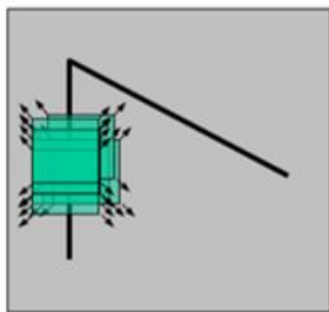
我们看到在图像中每个方向变化都很大的区域就是角点，一个早期的尝试是由 **Chris Harris & Mike Stephens** 在1998年的论文 **A Combined Corner and Edge Detector** 完成的。所以现在称之为 Harris 角点检测。

角点检测算法的基本思想：使用一个固定的窗口在图像上进行任意方向上的滑动，比较滑动前与滑动后两种情况，窗口中的像素灰度变化程度，如果存在任意方向上的滑动，都有着较大的灰度变化，那么我们可以认为该窗口中存在角点。



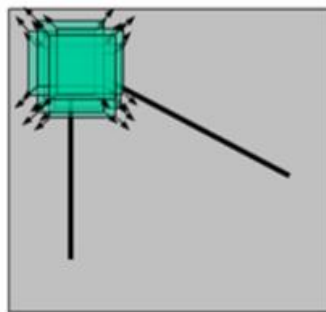
平面点：
x和y方向的变化不大

平面



平面点：
y方向的变化大

边界



平面点：
x和y方向的变化都大

角点



Chris Harris

角点检测

Harris角点检测

Harris角点检测的数学表达

我们先设 $I(x, y)$ 代表图像中 (x, y) 点的像素灰度值, $w(x, y)$ 代表以 (x, y) 为中心的Harris角点检测的滑动窗口, 即加权函数。

当窗口发生 $(\Delta x, \Delta y)$ 移动时, 那么滑动前与滑动后对应的窗口中的像素点灰度变化数学方法描述如下:

$$c(x, y; \Delta x, \Delta y) = \sum_{(u, v) \in W(x, y)} w(u, v) (I(u, v) - I(u + \Delta x, v + \Delta y))^2$$

减法: 获得平移前后的差异
平方: 获得变化的趋势, 无负数

角点检测

Harris角点检测

Harris角点检测的数学表达

$$c(x, y; \Delta x, \Delta y) = \sum_{(u,v) \in W(x,y)} w(u,v) (I(u,v) - I(u + \Delta x, v + \Delta y))^2$$

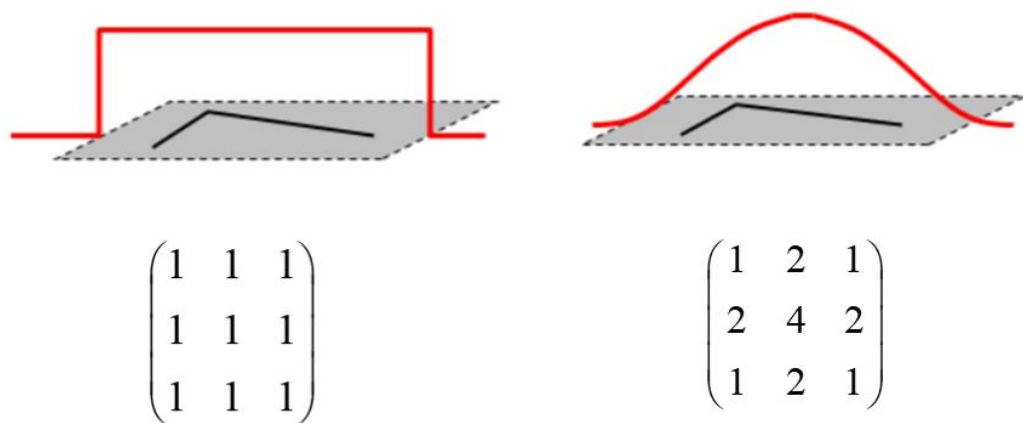
窗口 $\rightarrow$ $w(u,v)$

灰度值 $\rightarrow$ $I(u,v)$

减法: 获得平移前后的差异 $\rightarrow$ $I(u,v) - I(u + \Delta x, v + \Delta y)$

平方: 获得变化的趋势, 无负数 $\rightarrow$ $(I(u,v) - I(u + \Delta x, v + \Delta y))^2$

对于 $w(x,y)$ 窗口函数, 最简单情形就是窗口内的所有像素所对应的 w 权重系数均为1。但有时候, 我们会将 $w(x,y)$ 函数设定为以窗口中心为原点的二元正态分布, 如高斯分布。



如果窗口中心点是角点时, 移动前与移动后, 该点的灰度变化应该最为剧烈, 表示窗口移动时, 该点在灰度变化贡献较大; 而离窗口中心(角点)较远的点, 这些点的灰度变化几近平缓, 以示该点对灰度变化贡献较小, 那么我们自然想到使用二元高斯函数来表示窗口函数。



角点检测

Harris角点检测

Harris角点检测的数学表达

接下来是怎么解这个表达式了。其中最主要的是 $I(u, v) - I(u + \Delta x, v + \Delta y)$ 这一部分，我们采用高等数学中的泰勒公式。

$$c(x, y; \Delta x, \Delta y) = \sum_{(u, v) \in W(x, y)} w(u, v) (I(u, v) - I(u + \Delta x, v + \Delta y))^2$$

窗口 $\rightarrow$ $W(x, y)$

灰度值 $\rightarrow$ $I(u, v)$

减法: 获得平移前后的差异

平方: 获得变化的趋势, 无负数

一、问题的提出

一元函数的泰勒公式

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2 + \cdots + \frac{f^{(n)}(x_0)}{n!}(x - x_0)^n + \frac{f^{(n+1)}(x_0 + \theta(x - x_0))}{(n+1)!}(x - x_0)^{n+1} \quad (0 < \theta < 1).$$

意义: 可用 n 次多项式来近似表达函数 $f(x)$, 且误差是当 $x \rightarrow x_0$ 时比 $(x - x_0)^n$ 高阶的无穷小.

二、二元函数的泰勒公式

定理 设 $z = f(x, y)$ 在点 (x_0, y_0) 的某一邻域内连续且有直到 $n+1$ 阶的连续偏导数, $(x_0 + h, y_0 + h)$ 为此邻域内任一点, 则有

$$f(x_0 + h, y_0 + h) = f(x_0, y_0) + \left(h \frac{\partial}{\partial x} + k \frac{\partial}{\partial y} \right) f(x_0, y_0) + \frac{1}{2!} \left(h \frac{\partial}{\partial x} + k \frac{\partial}{\partial y} \right)^2 f(x_0, y_0) + \cdots + \frac{1}{n!} \left(h \frac{\partial}{\partial x} + k \frac{\partial}{\partial y} \right)^n f(x_0, y_0) + \frac{1}{(n+1)!} \left(h \frac{\partial}{\partial x} + k \frac{\partial}{\partial y} \right)^{n+1} f(x_0 + \theta h, y_0 + \theta k), \quad (0 < \theta < 1)$$



角点检测

Harris角点检测

Harris角点检测的数学表达

接下来是怎么解这个表达式了。其中最主要的是 $I(u, v) - I(u + \Delta x, v + \Delta y)$ 这一部分，我们采用高等数学中的泰勒公式。

$$c(x, y; \Delta x, \Delta y) = \sum_{(u, v) \in W(x, y)} w(u, v) (I(u, v) - I(u + \Delta x, v + \Delta y))^2$$

窗口 $\rightarrow$ $w(u, v)$

灰度值 $\rightarrow$ $I(u, v)$

减法: 获得平移前后的差异

平方: 获得变化的趋势, 无负数

$$I(u + \Delta x, v + \Delta y) = I(u, v) + I_x(u, v)\Delta x + I_y(u, v)\Delta y + O(\Delta x^2, \Delta y^2) \approx I(u, v) + I_x(u, v)\Delta x + I_y(u, v)\Delta y$$

其中, I_x, I_y 是 $I(x, y)$ 的偏导数

然后将近似结果带入, 然后再恒等变换处理一下:



角点检测

Harris角点检测

Harris角点检测的数学表达

接下来是怎么解这个表达式了。其中最主要的是 $I(u, v) - I(u + \Delta x, v + \Delta y)$ 这一部分，我们采用高等数学中的泰勒公式。

近似可得：

$$c(x, y; \Delta x, \Delta y) \approx \sum_w (I_x(u, v)\Delta x + I_y(u, v)\Delta y)^2 = [\Delta x, \Delta y] M(x, y) \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

将I和 ΔX 、 Δy 分开，
方便计算和理解

其中M:

$$M(x, y) = \sum_w \begin{bmatrix} I_x(x, y)^2 & I_x(x, y)I_y(x, y) \\ I_x(x, y)I_y(x, y) & I_y(x, y)^2 \end{bmatrix} = \begin{bmatrix} \sum_w I_x(x, y)^2 & \sum_w I_x(x, y)I_y(x, y) \\ \sum_w I_x(x, y)I_y(x, y) & \sum_w I_y(x, y)^2 \end{bmatrix} = \begin{bmatrix} A & C \\ C & B \end{bmatrix}$$

化简可得：

$$c(x, y; \Delta x, \Delta y) \approx A\Delta x^2 + 2C\Delta x\Delta y + B\Delta y^2$$

$$A = \sum_w I_x^2, B = \sum_w I_y^2, C = \sum_w I_x I_y$$

$$c(x, y; \Delta x, \Delta y) = \sum_{(u,v) \in W(x,y)} w(u,v) (I(u,v) - I(u + \Delta x, v + \Delta y))^2$$

窗口

灰度值

减法：获得平移前后的差异

平方：获得变化的趋势，无负数



角点检测

Harris角点检测

Harris角点检测的数学表达

接下来是怎么解这个表达式了。其中最主要的是 $I(u, v) - I(u + \Delta x, v + \Delta y)$ 这一部分，我们采用高等数学中的泰勒公式。

再分析最后的出来的这个公式，我们再把它简化一下：

其中M:

$$M(x, y) = \sum_w \begin{bmatrix} I_x(x, y)^2 & I_x(x, y)I_y(x, y) \\ I_x(x, y)I_y(x, y) & I_y(x, y)^2 \end{bmatrix} = \begin{bmatrix} \sum_w I_x(x, y)^2 & \sum_w I_x(x, y)I_y(x, y) \\ \sum_w I_x(x, y)I_y(x, y) & \sum_w I_y(x, y)^2 \end{bmatrix} = \begin{bmatrix} A & C \\ C & B \end{bmatrix}$$

化简可得:

$$c(x, y; \Delta x, \Delta y) \approx A\Delta x^2 + 2C\Delta x\Delta y + B\Delta y^2$$

M是个对称矩阵，则可以对角化成
 λ 就是矩阵的两个奇异值

$$\begin{pmatrix} \lambda_1 & \\ & \lambda_2 \end{pmatrix}$$

$$A = \sum_w I_x^2, B = \sum_w I_y^2, C = \sum_w I_x I_y$$

$$c(x, y; \Delta x, \Delta y) \approx \lambda_1 \Delta x^2 + \lambda_2 \Delta y^2$$

变成一个简单的椭圆公式了，我们先假设 $c = 1$ ，方便画图

$$c(x, y; \Delta x, \Delta y) = \sum_{(u, v) \in W(x, y)} w(u, v) (I(u, v) - I(u + \Delta x, v + \Delta y))^2$$

窗口

灰度值

减法：获得平移前后的差异

平方：获得变化的趋势，无负数

角点检测

Harris角点检测

Harris角点检测的数学表达

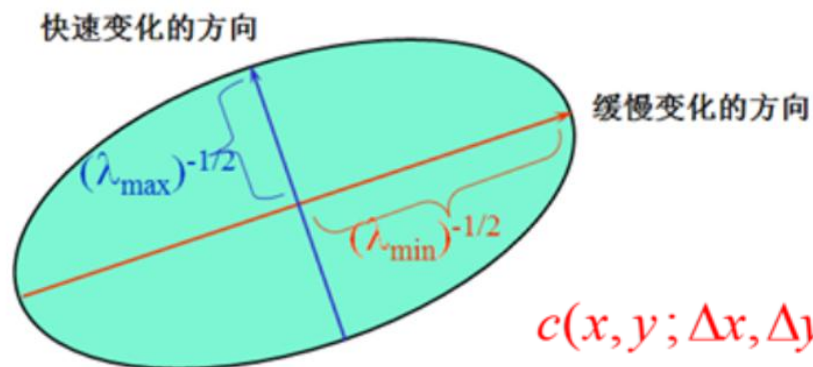
接下来是怎么解这个表达式了。其中最主要的是 $I(u, v) - I(u + \Delta x, v + \Delta y)$ 这一部分，我们采用高等数学中的泰勒公式。

二次项函数本质上就是一个椭圆函数，椭圆方程为：

$$[\Delta x, \Delta y] M(x, y) \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = 1$$

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

$$(a^2 = \lambda_1, b^2 = \lambda_2)$$



$$c(x, y; \Delta x, \Delta y) \approx \lambda_1 \Delta x^2 + \lambda_2 \Delta y^2$$

$$c(x, y; \Delta x, \Delta y) = \sum_{(u, v) \in W(x, y)} w(u, v) (I(u, v) - I(u + \Delta x, v + \Delta y))^2$$

窗口 $\rightarrow$ $W(x, y)$

灰度值 $\rightarrow$ $I(u, v)$

减法: 获得平移前后的差异 $\rightarrow$ $I(u, v) - I(u + \Delta x, v + \Delta y)$

平方: 获得变化的趋势, 无负数 $\rightarrow$ $(I(u, v) - I(u + \Delta x, v + \Delta y))^2$

接下来，我们就只要分析这个椭圆了：

如果在图像在窗口滑动过程中灰度变化较大，那是不是 $c(x, y; \Delta x, \Delta y)$ 的值就越大，那么这个椭圆也会变大？椭圆变大那么对应的 λ_1 和 λ_2 也会变大，然后就是比较 λ_1 和 λ_2 的变化程度。如果只有其中一个变化程度较大说明是边界。

角点检测

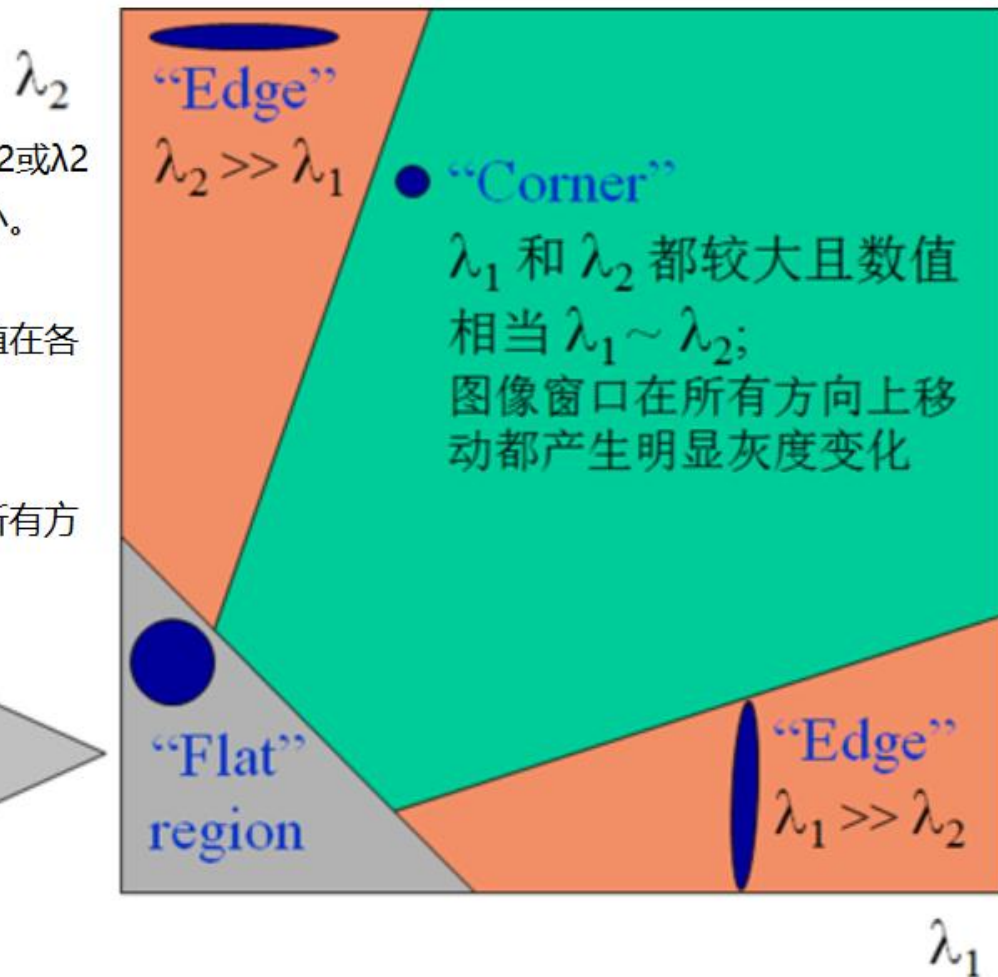
Harris角点检测

边界：一个特征值大，另一个特征值小， $\lambda_1 \gg \lambda_2$ 或 $\lambda_2 \gg \lambda_1$ 。自相关函数值在某一方向上大，在其他方向上小。

平面：两个特征值都小，且近似相等；自相关函数数值在各个方向上都小。

角点：两个特征值都大，且近似相等，自相关函数在所有方向都增大。

如果 λ_1 和 λ_2 都很小，
图像窗口在所有方向上
移动都无明显灰度变化





Harris角点检测

如何解出这两个特征值，并有效地进行量化评价呢？此时就引入角点响应值 R ，Harris角点检测算法就是对角点响应函数 R 进行阈值处理：

$$R > \text{threshold}$$

即提取 R 的局部极大值。这样选出的角点可能会在某一区域特别多，并且角点窗口 $w(x, y)$ 相互重合，为了能够更好地通过角点检测追踪目标，需要进行非极大值抑制（NMS）操作。

角点响应：

$$R = \det \mathbf{M} - \alpha (\text{trace} \mathbf{M})^2$$

其中：

$$\det \mathbf{M} = \lambda_1 \lambda_2 \quad \text{trace} \mathbf{M} = \lambda_1 + \lambda_2$$

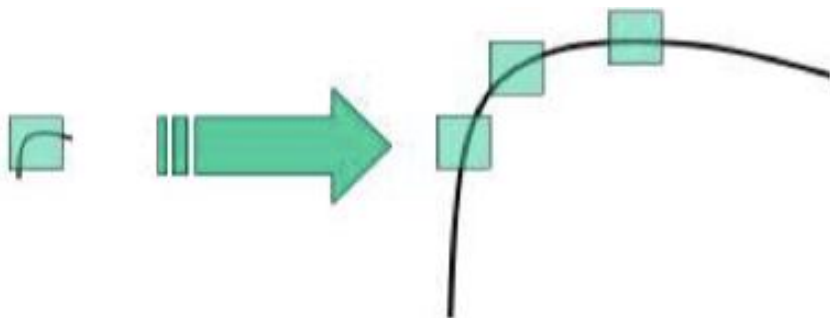
$\lambda_1 = 100, \lambda_2 = 101$	$R > 0$ 角点
$\lambda_1 = 0.1, \lambda_2 = 0.2$	$R \approx 0$ 平坦区域
$\lambda_1 = 100, \lambda_2 = 1$	$R < 0$ 边界

https://blog.csdn.net/qq_40274812

角点检测

Harris角点检测

注意：Harris 检测器具有旋转不变性，但不具有尺度不变性，也就是说尺度变化可能会导致角点变为边缘，如下图所示：



想要尺度不变特性的话，可以关注接下来要学的SIFT特征



图像局部特征点检测

图像特征可以包括颜色特征、纹理特征、形状特征以及局部特征点等。其中局部特点具有很好的稳定性，不容易受外界环境的干扰。

具体来说，局部特征点是**图像特征的局部表达**，它只能反映**图像上具有的局部特殊性**，所以它只适合于对图像进行**匹配，检索等应用**。对于图像理解则不太适合。而后者更关心一些全局特征，如颜色分布，纹理特征，主要物体的形状等。全局特征容易受到环境的干扰，光照，旋转，噪声等不利因素都会影响全局特征。相比而言，**局部特征点**，往往对应着图像中的一些**线条交叉，明暗变化的结构中**，受到的干扰也少。

而斑点与角点是两类局部特征点。**斑点通常是指与周围有着颜色和灰度差别的区域，如草原上的一棵树或一栋房子。它是一个区域，所以它比角点的噪能力要强，稳定性要好。**而角点则是图像中一边物体的拐角或者线条之间的交叉部分。

下面我们开始学习几个重要的局部特征点（斑点）检测算法：**SIFT、PCA-SIFT、SURF**

图像局部特征点检测

SIFT

SIFT (Scale-Invariant Feature Transform) 尺度不变特征变化。这是一种传统的图像特征提取此算法，由David G. Lowe于1999年在ICCV提出，在修改完善后，于2004年发表。

Distinctive Image Features from Scale-Invariant Keypoints, IJCV 2004.

SIFT算法专利于2000年申请，经过20年，已于2020年3月到期。opencv中已经重新集成了这种算法。

知 一代传奇SIFT算法专利到期 - 知乎

<https://zhuanlan.zhihu.com/p/114271966>

2020-3-18 · 谷歌学术显示SIFT 2004¹ 已被引用55841次 David Lowe 教授是一个很有商业嗅觉的学者，发明了SIFT算法后发现这可是个好东西，赶紧申请了专利 US6711293B1。专利申请于2000年3月6日，专利权人为 英属哥伦比亚大学。

C 【OpenCV】2020年关于SIFT算法专利版权问题的解决办法 ...

<https://blog.csdn.net/cleanlil/article/details/109561089>

2020-11-8 · 在opencv 3+的版本中，由于将SIFT，SURF这些有专利的算法单独提取到了opencv_contrib模块，因为官方给出的android sdk release版本中没有预先编译 opencv_contrib 至opencv库，要想在Android 中使用SURF，SIFT这些算法的C++实现，我们需要自己编译opencv_contrib opencv源码，从而获得opencv android sdk。

喜大普奔 | 专利到期！SIFT算法免费可用了_Lowe

www.sohu.com/a/380904326_823210

2020-3-17 · 谷歌学术显示SIFT 2004¹ 已被引用55841次 到今年3月6日，已满20年，专利权已经到期！SIFT已经成为全人类的公共技术，任何人和组织都可以免费使用！在今天，SIFT依然具有商业价值，这无疑是个好消息。



David G. Lowe
加州大学伯克利分校



SIFT

SIFT算法是一种基于局部兴趣点的算法，正如其名称那样，SIFT对于**图像的尺度与图像的旋转不敏感**，而且算法对于光照、噪声等影响也具有较好的鲁棒性。

SIFT算法主要分为四个步骤：

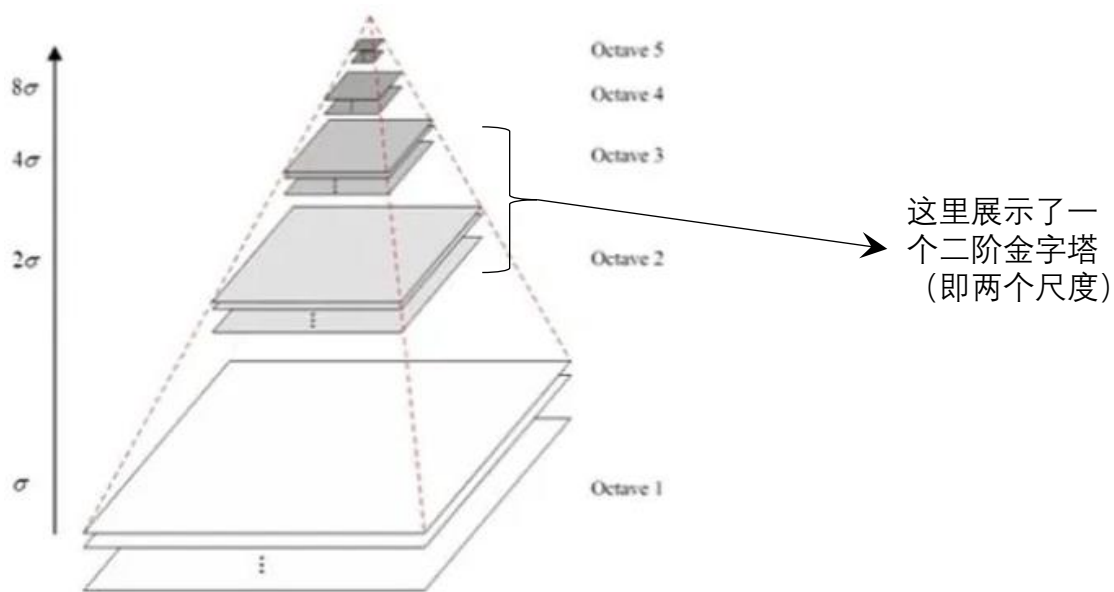
- 1.尺度空间极值检测 (Scale-space extrema detection)**：通过使用**高斯差分函数**来搜索所有尺度上的图像位置，识别出其中对于尺度和方向不变的潜在兴趣点。
- 2.关键点定位 (Keypoint localizatio)**：在每个候选位置上，利用一个拟合精细的模型确定位置和尺度，关键点的选择依赖于它们的稳定程度。
- 3.方向匹配 (Orientation assignment, 为每个关键点赋予方向)**：基于局部图像的梯度方向，为每个关键点位置分配一个或多个方向，后续所有对图像数据的操作都是基于相对关键点的方向、尺度和位置进行变换，从而获得了方向与尺度的不变性。
- 4.关键点描述符 (Keypoint descriptor)**：在每个关键点领域内，以选定的尺度计算局部图像梯度，这些梯度被变换成一种表示，这种表示允许比较大的局部形状的变形和光照变化。

图像局部特征点检测

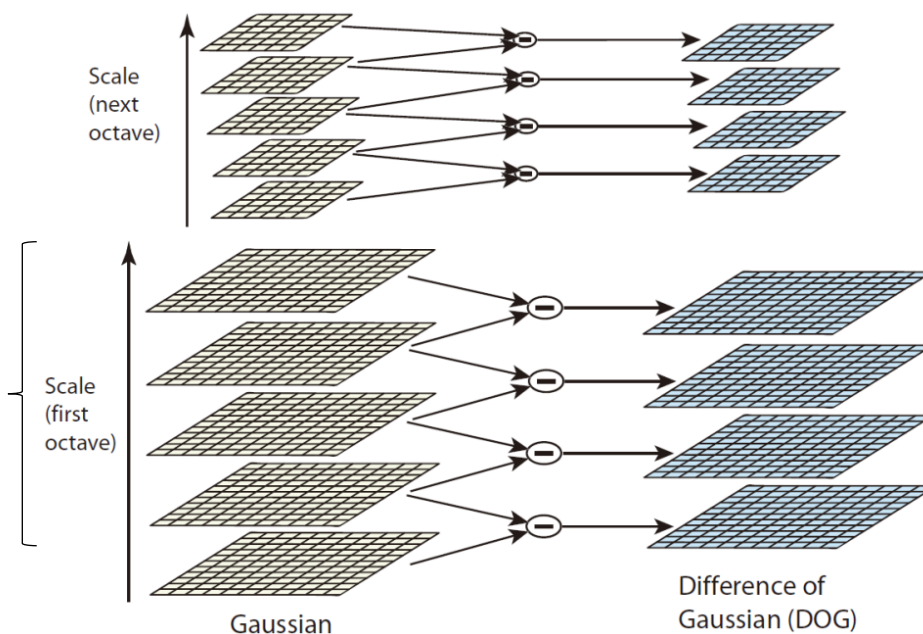
SIFT

1. 尺度空间极值检测 (Scale-space extrema detection)

$$G(u, v) = \frac{1}{2\pi\sigma^2} e^{-(u^2+v^2)/(2\sigma^2)}$$



同一尺度下
高斯卷积核
不同 (模糊
程度不同)

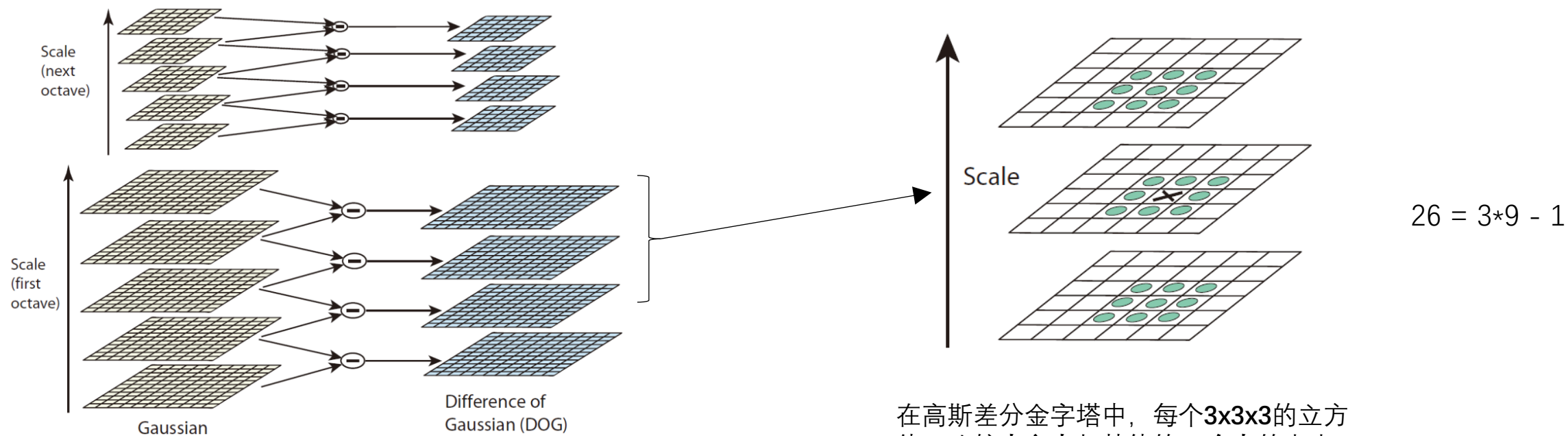


利用高斯核构建高斯金字塔，每个尺度层包含一组尺度的图像。这里不同组的特征金字塔的获取采用隔点取样获得。

在每个组内 (octave) 两两做差 (差分)，构建高斯差分金字塔。

SIFT

1. 尺度空间极值检测 (Scale-space extrema detection)

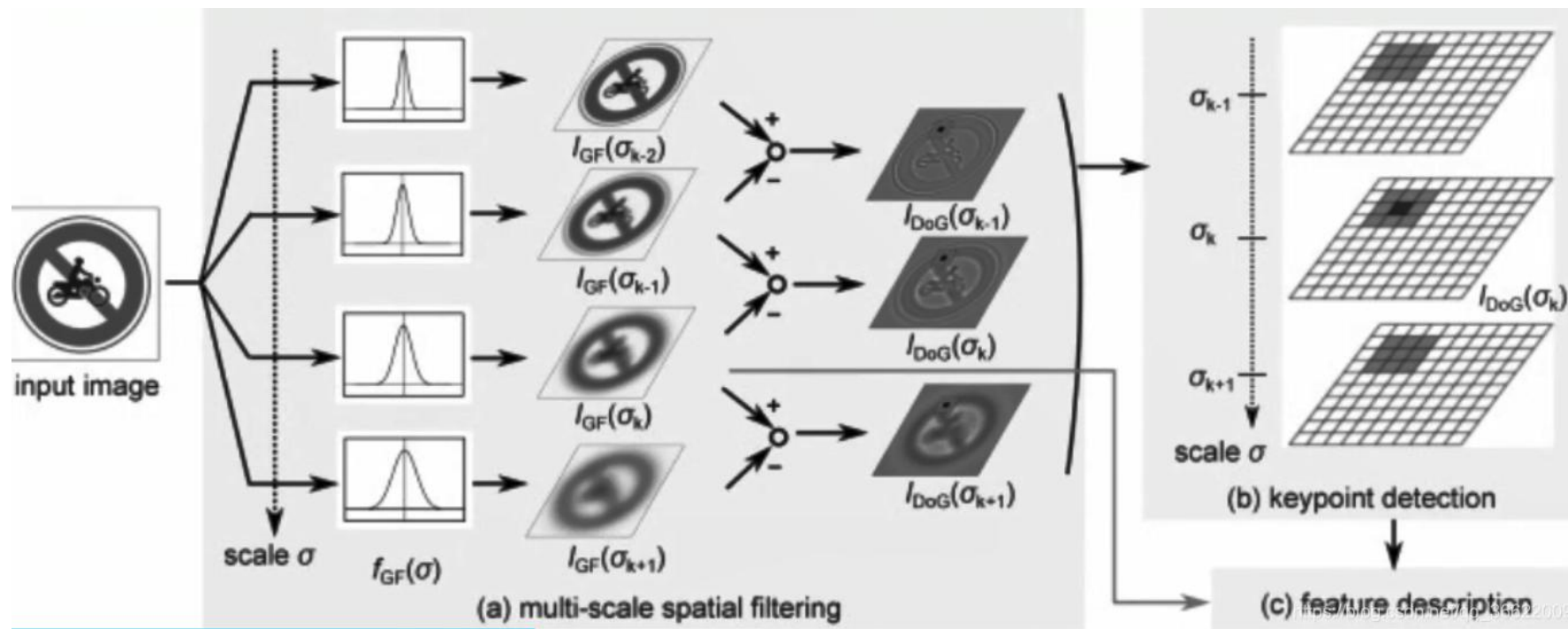


在高斯差分金字塔中，每个3x3x3的立方体，比较中心点与其他的26个点的大小，如果中心点最大或者最小，那么这个中心点就是极值点。

极值一般在边缘或灰度突变的地方

SIFT

1. 尺度空间极值检测 (Scale-space extrema detection)



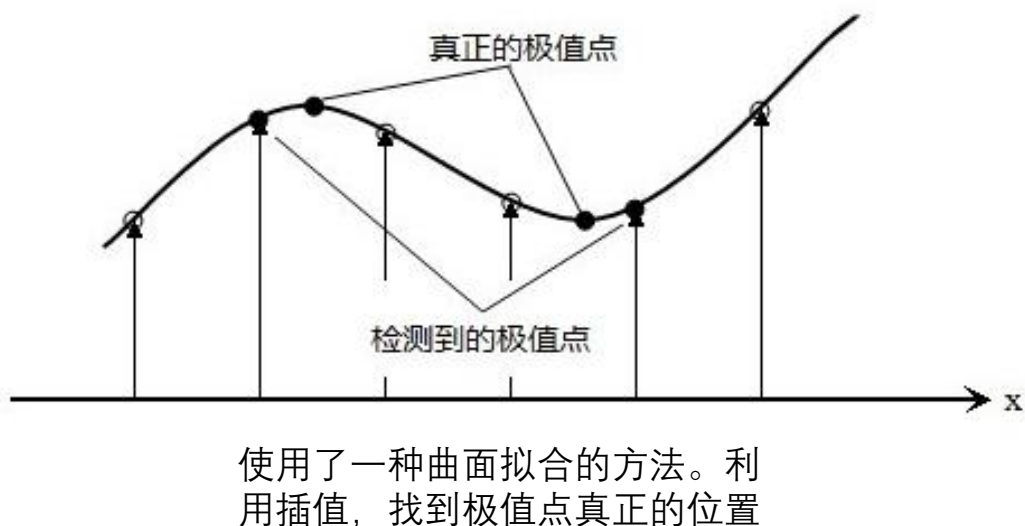
input image经过4个不同的高斯函数模糊后得到4张图（四个sigma变换），再进行两两差分，得到一些边界信息和极值点信息。对这些点进行比较，找出keypoint。

SIFT

2. 关键点定位 (Keypoint localization)

上边的步骤所检测到极值点是**离散空间的极值点**，这些极值点并不是十分准确，我们需要来精确确定关键点的位置和尺度，同时去除低对比度的关键点和不稳定的边缘点(因为DoG算子会产生较强的边缘响应)，从而增强稳定性。

离散空间的极值点并不是真正的极值点，正如下图所示，利用离散空间的极值点的插值可以得到**连续空间的极值点**。



因此，这里对于图像极值点进行二阶泰勒展开近似：

$$D(\mathbf{x}) = D + \frac{\partial D^T}{\partial \mathbf{x}} \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x}$$

求导，让其为0，求得连续空间的极值点：

$$\hat{\mathbf{x}} = -\frac{\partial^2 D}{\partial \mathbf{x}^2}^{-1} \frac{\partial D}{\partial \mathbf{x}}$$

$$D(\hat{\mathbf{x}}) = D + \frac{1}{2} \frac{\partial D^T}{\partial \mathbf{x}} \hat{\mathbf{x}}.$$

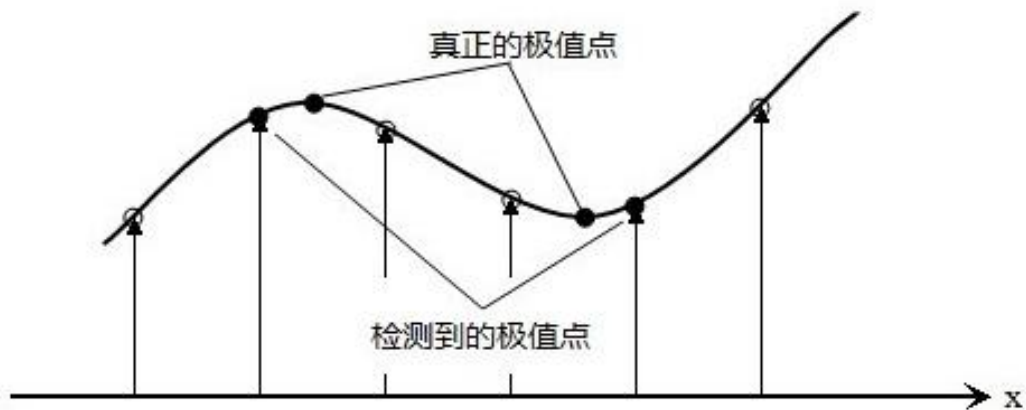
将极值点代入上述泰勒近似公式，得到在极值点位置的值：

SIFT

2. 关键点定位 (Keypoint localization)

上边的步骤所检测到极值点是**离散空间的极值点**，这些极值点并不是十分准确，我们需要来精确确定关键点的位置和尺度，同时去除低对比度的关键点和不稳定的边缘点(因为DoG算子会产生较强的边缘响应)，从而增强稳定性。

离散空间的极值点并不是真正的极值点，正如下图所示，利用离散空间的极值点的插值可以得到**连续空间的极值点**。



由此，即可得到真正的连续空间的极值点。

然后去除低对比度以及**边缘效应的极值点**（这部分的细节可以看原始论文），就得到最终需要保留的所有极值点。



SIFT

2. 关键点定位 (Keypoint localization)

上边的步骤所检测到极值点是**离散空间的极值点**，这些极值点并不是十分准确，我们需要来精确确定关键点的位置和尺度，同时去除低对比度的关键点和不稳定的边缘点(因为DoG算子会产生较强的边缘响应)，从而增强稳定性。

边缘效应的去除，采用Hessian矩阵（海瑟矩阵）的特征值，Hessian矩阵实际为二阶偏导数。在高斯差分图像中，如果其边缘特征点，那么其垂直于边缘的方向，值变化较大（对应于Hessian矩阵中较大的特征值），沿边缘方向，值变化较小（对应于Hessian矩阵中较小的特征值）。因此采用Hessian矩阵中的较大特征值与较小特征值的比例超过阈值的特征点去除，就去除了边缘效应。

$$\mathbf{H} = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}$$

$$\text{Tr}(\mathbf{H}) = D_{xx} + D_{yy} = \alpha + \beta,$$

$$\text{Det}(\mathbf{H}) = D_{xx}D_{yy} - (D_{xy})^2 = \alpha\beta.$$

$$\frac{\text{Tr}(\mathbf{H})^2}{\text{Det}(\mathbf{H})} < \frac{(r+1)^2}{r}.$$

论文中取 $r = 10$.

SIFT

2. 关键点定位 (Keypoint localization)

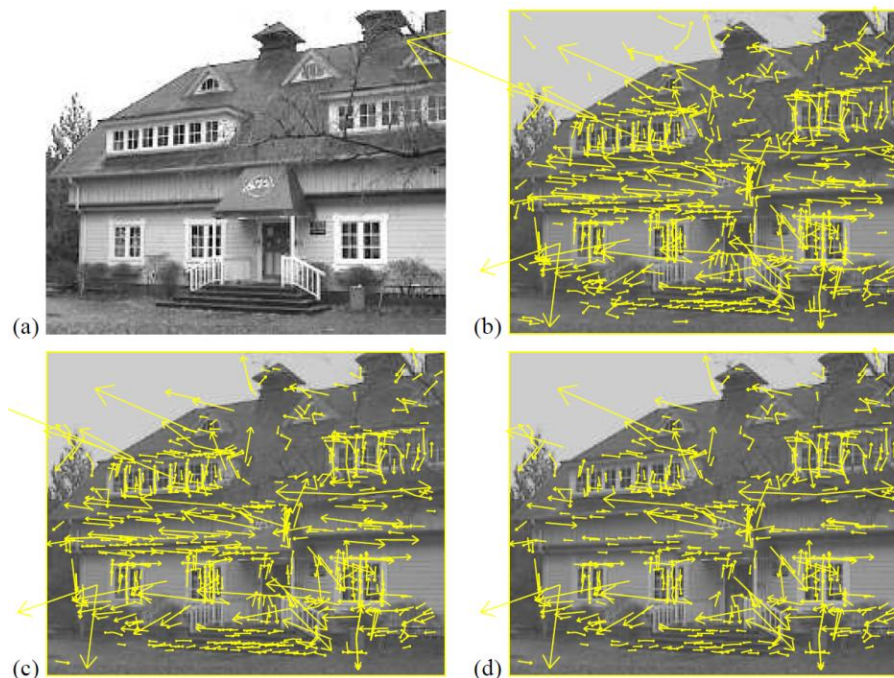
上边的步骤所检测到极值点是**离散空间的极值点**，这些极值点并不是十分准确，我们需要来精确确定关键点的位置和尺度，同时去除低对比度的关键点和不稳定的边缘点(因为DoG算子会产生较强的边缘响应)，从而增强稳定性。

(a)图是原图像，

(b)图是直接进行关键点检测得到的效果图

(c)图是舍弃对比度较小的极值点得到的效果图

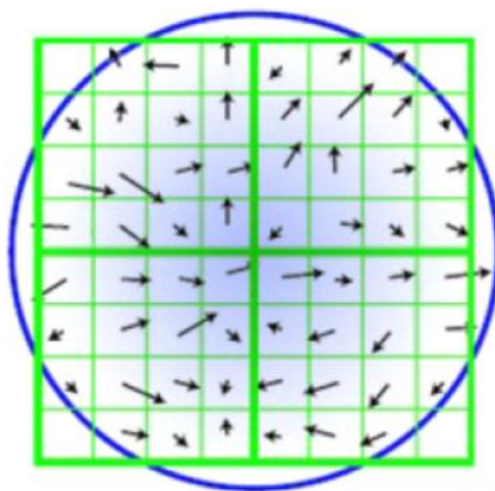
(d)图是去除边缘效应极值点后得到的效果图



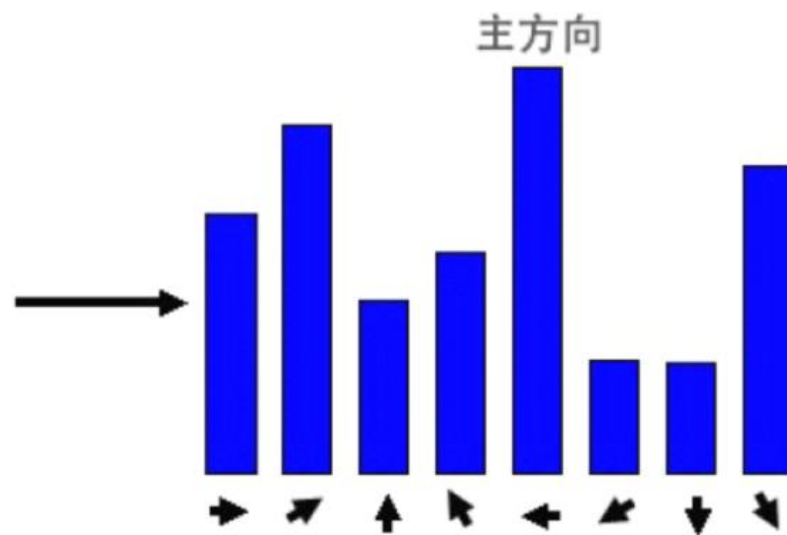
SIFT

3. 方向赋值 (Orientation assignment, 为每个关键点赋予方向)

对于每个极值点，统计以该特征点所在的高斯图像的尺度的1.5倍为半径的圆内的所有的像素的梯度方向及其梯度幅值。得到右图所示的直方图
(论文中以每10度为一个范围，也就是有36个方向)。直方图峰值所对应的方向为主方向，任何大于峰值80%的方向为特征点的辅方向。



半径为: $3 \times 1.5 \times \sigma$
上面这些箭头的方向就是梯度的方向
箭头的长度代表了梯度赋值大小
长度越长, 赋值越大



10度一柱, 共 $360^\circ / 10 = 36$ 柱
(注: 这里的示意图只给了8个方向)

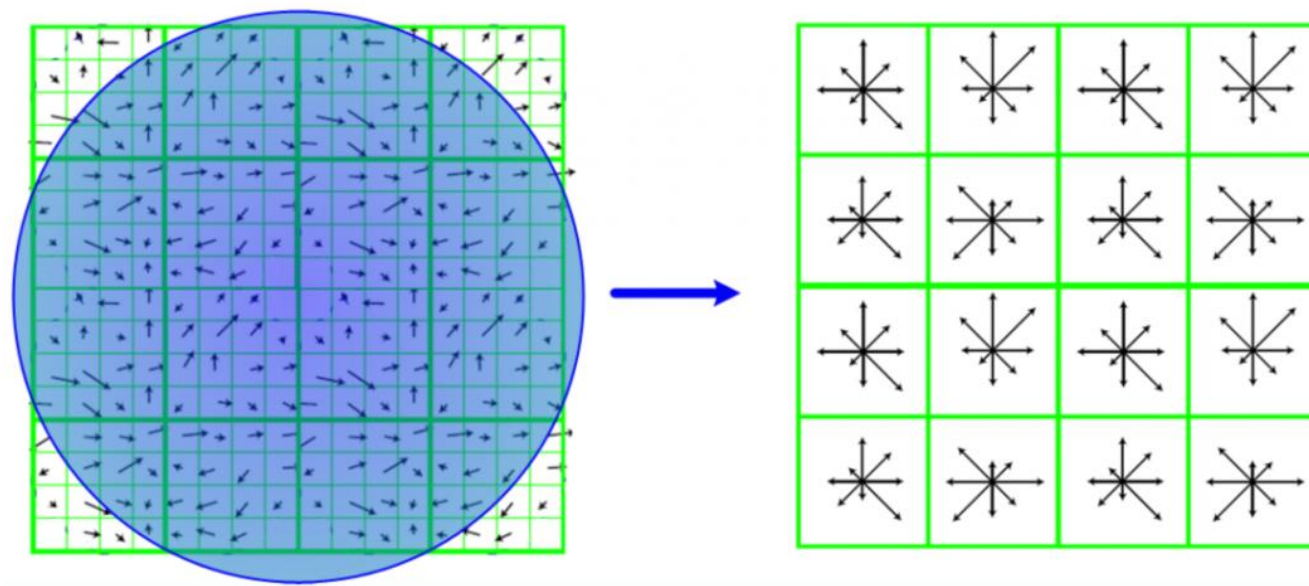
SIFT

4. 关键点描述符 (Keypoint descriptor)

前边三个步骤，我们找到了所有的特征点的位置，并且每个特征点都有方向，尺度信息。接下来就是计算在局部区域内这些特征点的描述符（描述子）。

左边是图像梯度图像，右边是关键点描述符。描述符与特征点所在的尺度图像有关，因此在**某个高斯尺度图像**上，以特征点为圆心，将其附近邻域划分为**4x4个子区域**，每个区域统计特征点的方向以及尺度，每个子区域获得8个方向的梯度信息。

因此，每个特征点共有 **$4 \times 4 \times 8 = 128$** 维的特征。

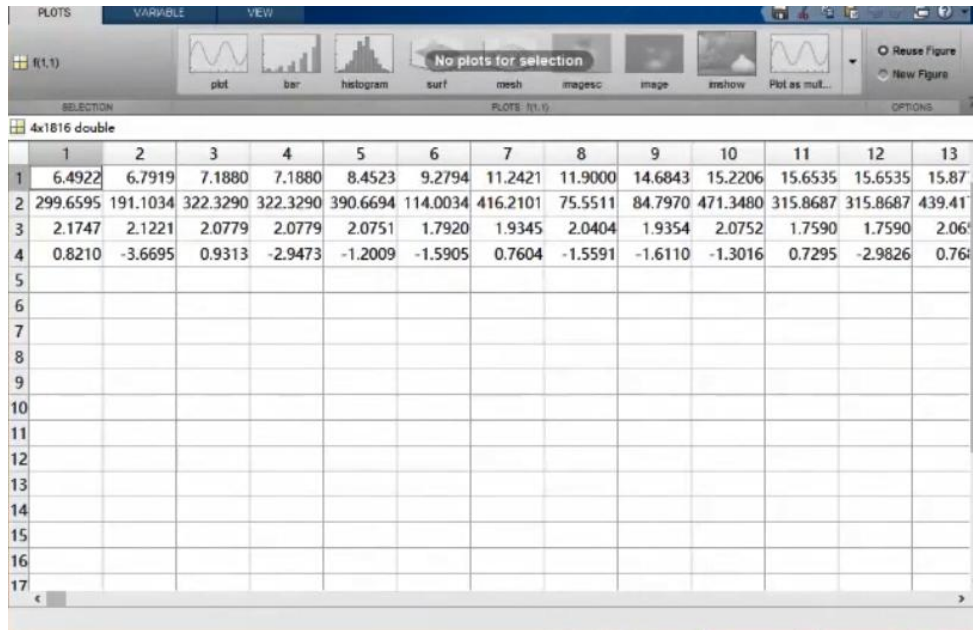


SIFT

4. 关键点描述符 (Keypoint descriptor)



SIFT特征点展示 (x, y, scale, rotation) , 其中
 x,y – 对应上图的特征点位置
 scale – 上图圆越大, 证明在大尺度图像上找到的特征点
 rotation – 看圆中的直线所指主方向



SIFT特征点展示 (x, y, scale, rotation) , 其中
 x,y – 第二步提取的位置信息
 scale – 第一步提取的高斯金字塔的尺度信息
 rotation – 第三步提取的主方向信息

PCA-SIFT

PCA-SIFT是对传统SIFT算法的改进，由Yan Ke等人在《PCA-SIFT: A More Distinctive Representation for Local Image Descriptors》中提出，论文中采用PCA(Principal Component Analysis)把SIFT特征描述子从128维降到了20维，优化了描述子占用的内存，同时提高了匹配的准确性。



Yan Ke

Clobotics
Verified email at yanke.org - [Homepage](#)
Computer vision Machine learning Robotics

FOLLOW

TITLE	CITED BY	YEAR
PCA-SIFT: A more distinctive representation for local image descriptors Y Ke, R Sukthankar Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision ...	5339	2004



Rahul Sukthankar

[Google Research](#)
Verified email at google.com - [Homepage](#)
computer vision machine learning

FOLLOWING

TITLE	CITED BY	YEAR
Large-scale video classification with convolutional neural networks A Karpathy, G Toderici, S Shetty, T Leung, R Sukthankar, L Fei-Fei Proceedings of the IEEE conference on Computer Vision and Pattern ...	7989	2014
PCA-SIFT: A more distinctive representation for local image descriptors Y Ke, R Sukthankar Proceedings of the 2004 IEEE Computer Society Conference on Computer Vision ...	5339	2004



PCA-SIFT

PCA-SIFT是对传统SIFT算法的改进，由Yan Ke等人在《PCA-SIFT: A More Distinctive Representation for Local Image Descriptors》中提出，论文中采用PCA（Principal Component Analysis）把SIFT特征描述子从128维降到了20维，优化了描述子占用的内存，同时提高了匹配的准确性。

1,2,3步骤与SIFT原理的相同，第四步(PCA):

注意：这里的39是因为最外层像素不计算偏导数

- (1) 对特征点确定一个大小为 41×41 的邻域，旋转这个邻域到主方向；
- (2) 计算邻域内像素点的水平梯度与垂直梯度，这样每个特征点确定了一个大小为 $39 \times 39 \times 2 = 3042$ 维的特征描述子；
- (3) 原论文中对同一类图像集中采集了21000个特征点，这样够成了一个原始特征矩阵M，矩阵大小 3042×21000 ，计算矩阵的协方差矩阵N；
- (4) 计算协方差矩阵N的特征向量，根据特征值的大小排序，选择对应的前n个特征向量(论文中采用 $n=20$)，构成投影矩阵T；
- (5) 对新的特征描述子向量，乘以投影矩阵T，得到3042维降到n维的特征向量；

实际上，步骤(3),(4)是在之前计算好，即投影矩阵是通过同一类图像集采用PCA原理提前计算出。从图像中特征点计算得到3042维的特征描述子时，只需要与投影矩阵相乘，即可达到降维。关于投影矩阵的n的选择，可以根据需要固定n的值，也可根据协方差矩阵的特征值能量值百分比，自动确定n的大小。论文中采用 $n=20$ 效果最佳。



PCA-SIFT

PCA-SIFT是对传统SIFT算法的改进，由Yan Ke等人在《PCA-SIFT: A More Distinctive Representation for Local Image Descriptors》中提出，论文中采用PCA(Principal Component Analysis)把SIFT特征描述子从128维降到了20维，优化了描述子占用的内存，同时提高了匹配的准确性。

- (1) We extract a 41×41 patch at the given scale,
- (2) Centered over the keypoint, and rotated to align its dominant orientation to a canonical direction.

PCA-SIFT can be summarized in the following steps:

- (3) pre-compute an eigenspace to express the gradient images of local patches;
- (4) given a patch, compute its local image gradient;
- (5) project the gradient image vector using the eigenspace to derive a compact feature vector.

This feature vector is significantly smaller than the standard SIFT feature vector, and can be used with the same matching algorithms. The Euclidean distance between two feature vectors is used to determine whether the two vectors correspond to the same keypoint in different images.

PCA-SIFT

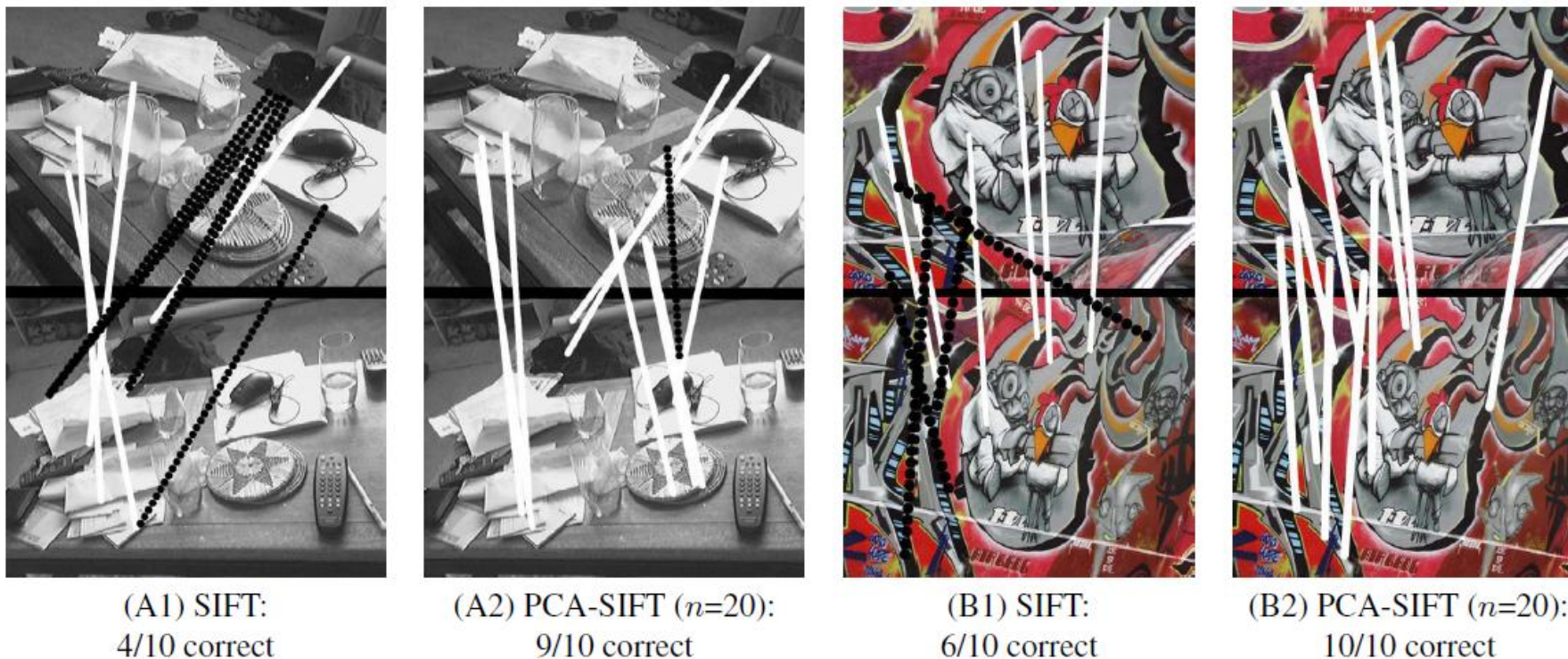


Figure 3: A comparison between SIFT and PCA-SIFT ($n=20$) on some challenging real-world images taken from different viewpoints. (A) is a photo of a cluttered coffee table; (B) is a wall covered in Graffiti from the INRIA Graffiti dataset. The top ten matches are shown for each algorithm: solid white lines denote correct matches while dotted black lines show incorrect ones.



图像局部特征点检测

SURF

SURF算法是对SIFT算法的改进，主要是提升了运算的速度。



Herbert Bay

Earkick
Verified email at earkick.com - [Homepage](#)
[Computer Vision](#) [Machine Learning](#) [Mixed Reality](#)

FOLLOW

TITLE

CITED BY

YEAR

Surf: Speeded up robust features

H Bay, T Tuytelaars, L Van Gool
Computer Vision-ECCV 2006: 9th European Conference on Computer Vision, Graz ...

36266 *

2006

Herbert Bay

Entrepreneur and Investor - Expert in Computer Vision and Machine Learning - Adventurer

Entrepreneur and Investor

- Co-founder of two Swiss/US AI companies:
 - [Earkick](#), Current Position, tracking mental health.
 - [kooaba](#), the first cloud-based image-recognition platform - exit 2014 to Qualcomm
 - [Shortcut](#), an AI-powered recipe and cooking app
- Driving innovation at companies such as Ifolor where I led [Captural](#) as their CEO and co-initiator, an AI-enabled photo-book app
- Investing in AI companies such as [Let's Enhance](#)
- Coaching AI companies such as [ethonAI](#)

Board Member and Advisor

- Member of the board of directors at [Ifolor](#)
- Co-founder and member of the board of directors at [47nord](#)
- Investor and advisor to US-based startup company [Let's Enhance](#)
- Previously advising Tokyo-based AR / SLAM startup company [Kudan](#)
- Previously advising Moscow-based mapping startup company [Maps.me](#) (acquired in Nov 2014 by Mail.Ru)

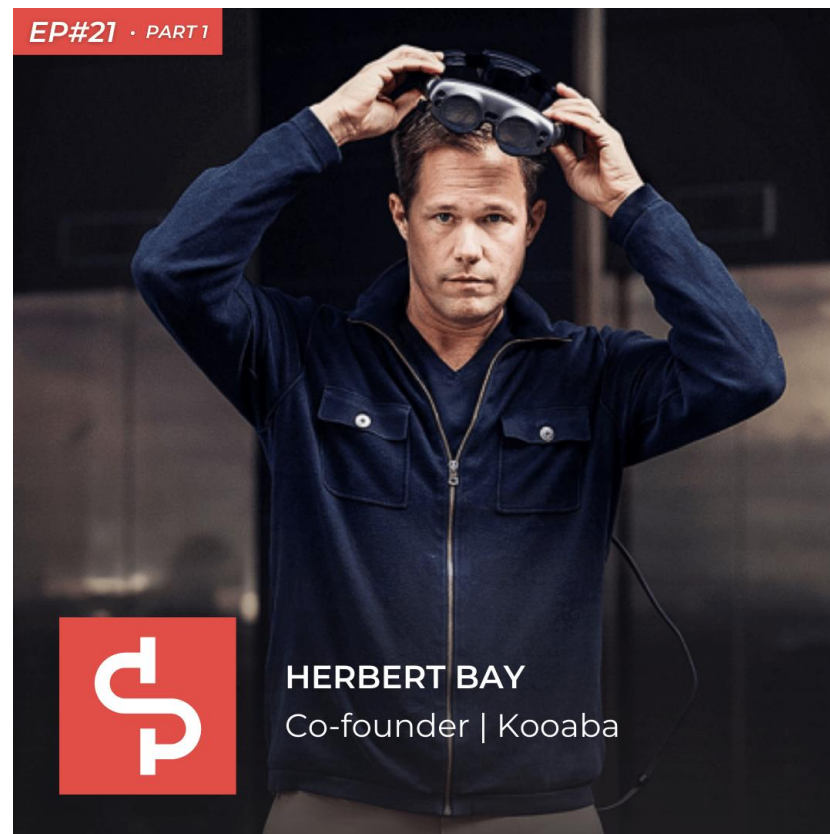
Computer Vision and Machine Learning

- Lead-inventor of the [SURF algorithm](#) that was cited over **29,000** times in scientific publications
- Received the 2016 [ECCV - Koenderink Prize](#) given to fundamental contributions in computer vision that stood the test of time.
- Received the 2011 [CVIU Most Cited Paper Award](#), and many more...
- Building and leading Computer Vision and Machine Learning teams at companies such as [Magic Leap](#)

Adventurer

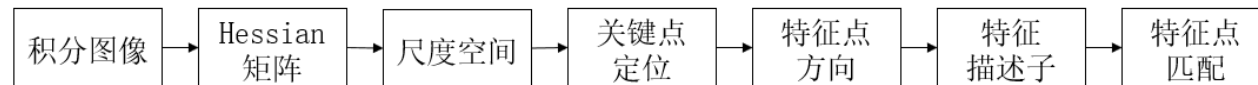
- Crossed the Atlantic and the Pacific ocean on a sailboat with my wife and our two wonderful boys
- More adventures coming soon, stay tuned :-)

EP#21 · PART 1



HERBERT BAY
Co-founder | Kooaba

SURF

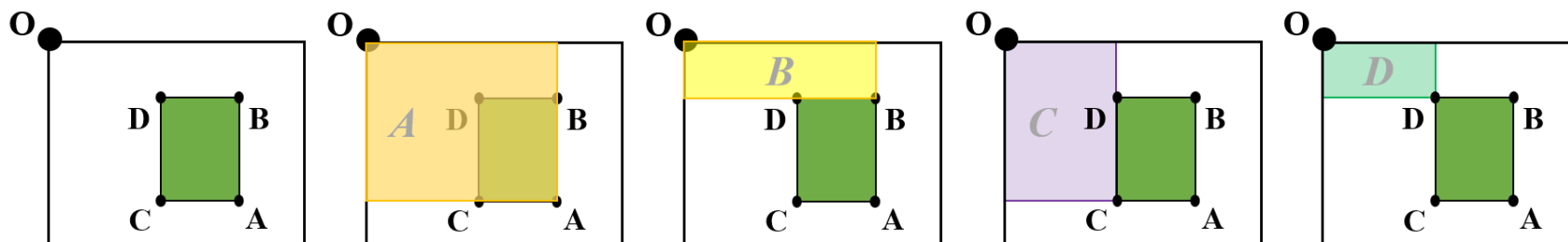


1. 积分图像

SURF算法中要用到积分图像的概念。借助积分图，图像与高斯二阶微分模板的滤波转化为对积分图像的加减运算，从而在特征点的检测时大大缩短了搜索时间。

积分图主要的思想是将图像从起点开始到各个点所形成的矩形区域像素之和作为一个数组的元素保存在内存中，当要计算某个区域的像素和时可以直接索引数组的元素，不用重新计算这个区域的像素和，从而加快了计算。

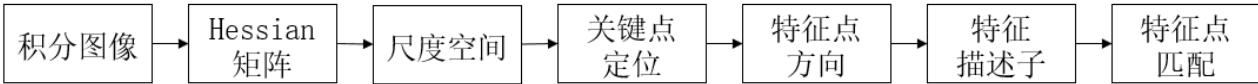
求取积分图时，对图像所有像素遍历一遍，得到积分图后，计算任何矩形区域内的像素灰度和只需进行三次加减运算，如下图所示。



图中绿色区域的面积为： $A - B - C + D$ ，因为在减法的过程中，多减了一次D，所以最后要加上一个D。



SURF



2. Hessian矩阵近似

图像点的二阶微分Hessian矩阵的行列式 (Determinant of Hessian, DoH) 的极大值, 可用于图像的斑点检测 (Blob Detection)。其中对于Hessian矩阵定义, 给定图像 I 中的一个点 $x(i, j)$, 在点 x 处, 尺度为 σ 的Hessian矩阵 $H(x, \sigma)$ 的公式如下:

$$H(x, y, \sigma) = \begin{pmatrix} L_{xx} & L_{xy} \\ L_{xy} & L_{yy} \end{pmatrix}$$

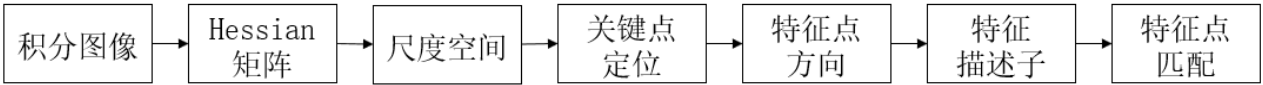
其中 L_{xx} , L_{yy} , L_{xy} 是高斯二阶微分算子 $\frac{\partial^2 g}{\partial x^2}$, $\frac{\partial^2 g}{\partial y^2}$, $\frac{\partial^2 g}{\partial x \partial y}$ 在 x 处与原图像的卷积, Hessian矩阵的行列式值DoH为:

$$\det H = L_{xx} L_{yy} - L_{xy}^2$$

DoH反映了图像局部的纹理或结构信息, 对图像中细长结构的斑点有较好的抑制作用。DoH在利用二阶微分算子对图像进行斑点检测时, 都需要利用高斯滤波平滑图像、抑制噪声, 检测过程主要分为以下两步:

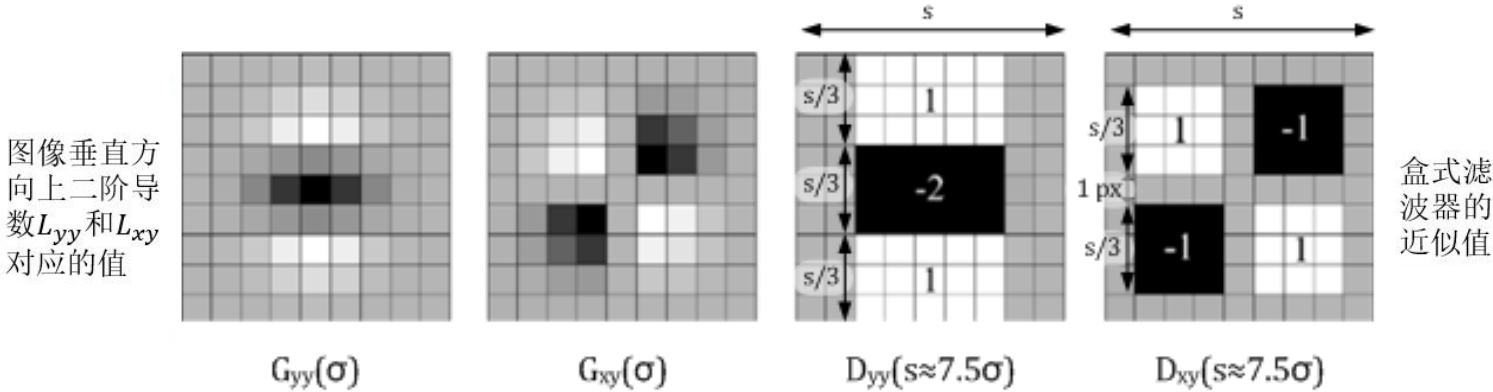
- 1) 使用不同的 σ 生成 $\frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$ 或 $\frac{\partial^2 g}{\partial x^2}$, $\frac{\partial^2 g}{\partial y^2}$, $\frac{\partial^2 g}{\partial x \partial y}$ 高斯卷积模板, 并对图像进行卷积运算。
- 2) 在图像的位置空间和尺度空间搜索LoG或DoH的峰值, 并进行非极大值抑制, 精确定位到图像极值点。

SURF



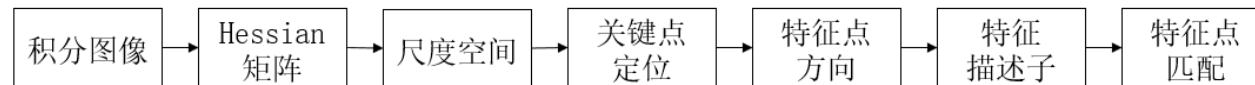
2. Hessian矩阵近似

为了将模板与图像的卷积转化为**盒式滤波器运算**，并能够使用积分图，需要对高斯二阶微分模板进行简化，使得简化后的模板只是由几个矩形区域组成，矩形区域内填充同一值，如下图所示，简化模板中白色区域的值为1，黑色区域的值为-1或-2（由相对面积决定），灰色区域的值为0。





SURF



2. Hessian矩阵近似

对于 $\sigma = 1.2$ 的高斯二阶微分滤波，设定模板的尺寸为 9×9 的大小，并用它作为最小尺度空间值对图像进行滤波和斑点检测。使用 D_{xx} 、 D_{xy} 和 D_{yy} 表示模板与图像进行卷积的结果。所以此时可以将Hessian矩阵的行列式作如下的简化：

$$\begin{aligned}
 Det(H_{approx}) &= L_{xx}L_{yy} - L_{xy}L_{xy} \\
 &= D_{xx} \frac{L_{xx}}{D_{xx}} D_{yy} \frac{L_{yy}}{D_{yy}} - D_{xy} \frac{L_{xy}}{D_{xy}} D_{xy} \frac{L_{xy}}{D_{xy}} \\
 &= D_{xx} D_{yy} - (w D_{xy})^2
 \end{aligned}$$

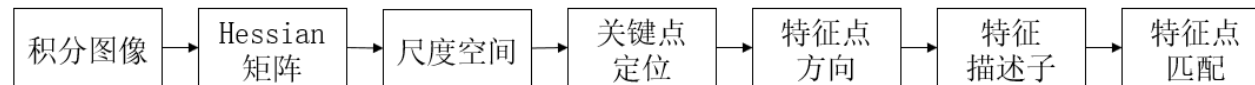
滤波器响应的相关权重 w 是为了平衡Hessian行列式的表示式。这是为了保持高斯核与近似高斯核的一致性。

$$w = \frac{|L_{xy}(\sigma)|_F |D_{xx}(\sigma)|_F}{|L_{xx}(\sigma)|_F |D_{xy}(\sigma)|_F} = 0.912$$

其中 $|\cdot|_F$ 为Frobenius范数（即F-范数）。理论上来说对于不同的 σ 的值和对应尺寸的模板尺寸， w 值是不同的，但为了简化起见，可以认为它是同一个常数0.9。在实际计算滤波响应值时，需要使用模板中盒式区域的面积进行归一化处理，以保证一个统一的Frobenius范数能适应所有的滤波尺寸。



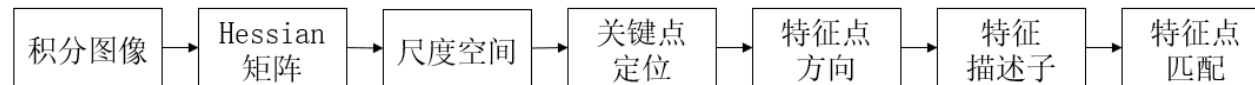
SURF



2. Hessian矩阵近似

使用近似的Hessian矩阵行列式来表示一个图像中某一点处的斑点响应值，遍历图像中的所有像素，便形成了在某一尺度下斑点检测的响应图像。使用不同的模糊尺度和模板尺寸，便形成了多尺度斑点响应的金字塔图像，利用这一金字塔图像，可以进行斑点响应极值点的搜索定位，其过程与SIFT算法类似。

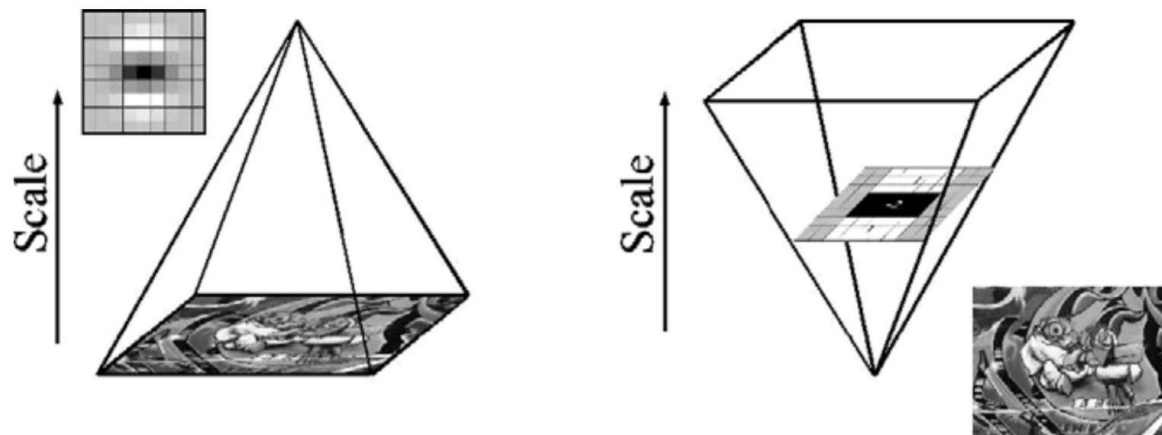
SURF



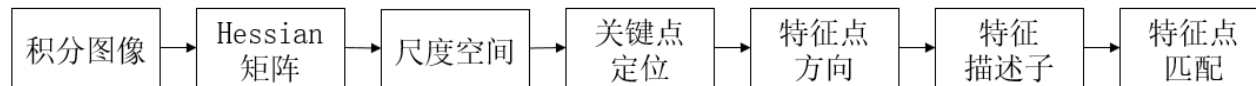
3.尺度空间表示

要想检测不同尺度的极值点，必须建立图像的尺度空间金字塔。一般的方法是通过采用不同 σ 的高斯函数，对图像进行平滑滤波，然后重采样获得更高一层的金字塔图像。SIFT算法中是通过相邻两层高斯金字塔图像相减得到DoG图像，然后在DoG金字塔图像上进行特征点检测。

与SIFT特征不同的是，**SURF算法不需要通过降采样的方式得到不同尺寸大小的图像建立金字塔，而是借助于盒式（方框）滤波和积分图像，不断增大盒式滤波模板，通过积分图快速计算盒式滤波的响应图像。**然后在响应图像上采用非极大值抑制，检测不同尺度的特征点。SIFT算法的LoG金字塔和SURF算法的近似DoH金字塔如下图所示：



SURF

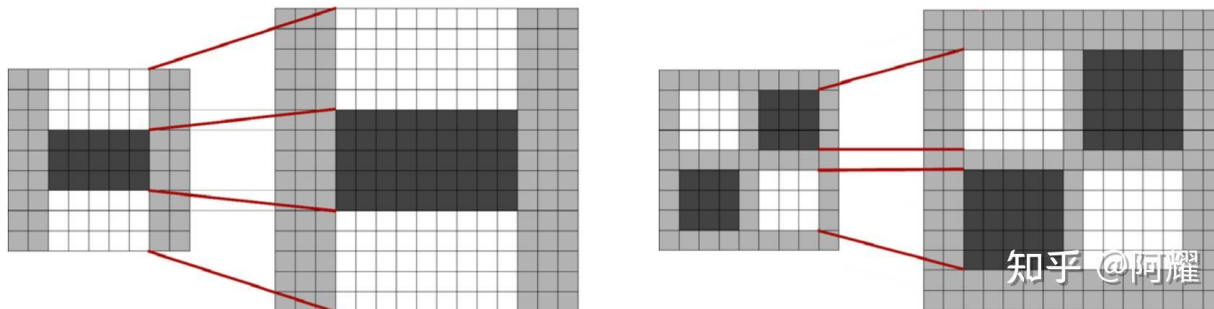


3.尺度空间表示

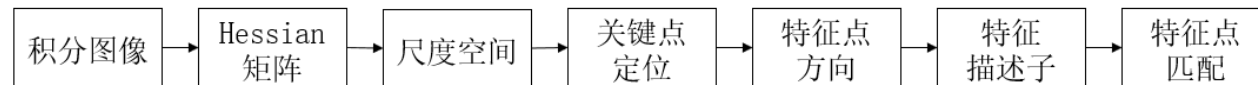
如前所述，使用 9×9 的模板对图像滤波，其结果作为最初始的尺度空间层，后续层将通过逐步增大滤波模板尺寸，以及放大后的模板与图像卷积得到。由于采用了盒式滤波和积分图，滤波过程并不随着滤波模板尺寸的增大而增加运算量。

在建立盒式滤波金字塔时，与SIFT算法类似，需要将尺度空间划分为若干组。每组又由若干固定层组成，包括不同尺寸的滤波模板对同一输入图像进行滤波得到的一系列响应图。由于积分图像的离散特性，两个相邻层之间的最小尺度变化量，是由高斯二阶微分滤波模板在微分方向上对正负斑点响应长度 l_0 决定的，它是盒式滤波模板尺寸的 $1/3$ 。对于 9×9 的滤波模板， l_0 为3。

下一层的响应长度至少应该在 l_0 的基础上增加2个像素，以保证一边一个像素，即 $l_0 = 5$ ，这样模板的尺寸为 15×15 ，如下图所示。依次类推，可以得到一个尺寸逐渐增大的模板序列，尺寸分别为 9×9 、 15×15 、 21×21 、 27×27 。显然，第一个模板和最后一份模板产生的Hessian响应图像只作为比较用，而不会产生最后的响应极值。如下图所示，黑色、白色区域的长度增加偶数个像素，以保证一个中心像素的存在。



SURF

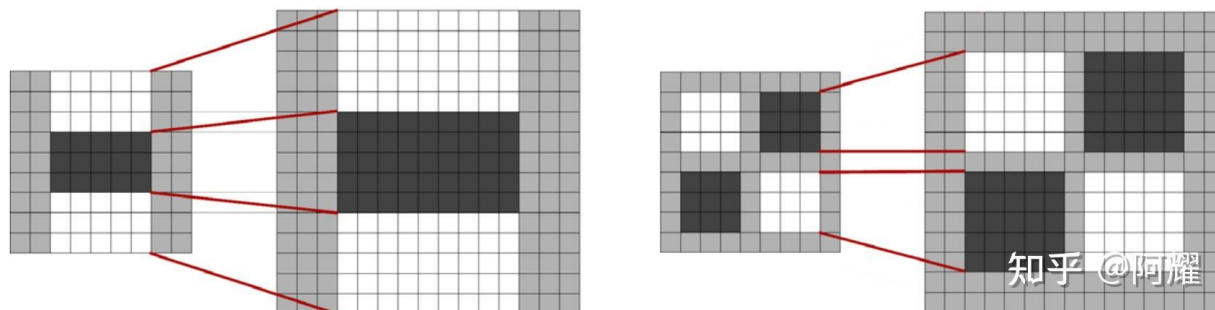


3.尺度空间表示

如前所述，使用 9×9 的模板对图像滤波，其结果作为最初尺度空间层，后续层将通过逐步增大滤波模板尺寸，以及放大后的模板与图像卷积得到。由于采用了盒式滤波和积分图，滤波过程并不随着滤波模板尺寸的增大而增加运算量。

在建立盒式滤波金字塔时，与SIFT算法类似，需要将尺度空间划分为若干组。每组又由若干固定层组成，包括不同尺寸的滤波模板对同一输入图像进行滤波得到的一系列响应图。由于积分图像的离散特性，两个相邻层之间的最小尺度变化量，是由高斯二阶微分滤波模板在微分方向上对正负斑点响应长度 l_0 决定的，它是盒式滤波模板尺寸的 $1/3$ 。对于 9×9 的滤波模板， l_0 为3。

下一层的响应长度至少应该在 l_0 的基础上增加2个像素，以保证一边一个像素，即 $l_0=5$ ，这样模板的尺寸为 15×15 ，如下图所示。依次类推，可以得到一个尺寸逐渐增大的模板序列，尺寸分别为 9×9 、 15×15 、 21×21 、 27×27 。显然，第一个模板和最后一份模板产生的Hessian响应图像只作为比较用，而不会产生最后的响应极值。如下图所示，黑色、白色区域的长度增加偶数个像素，以保证一个中心像素的存在。

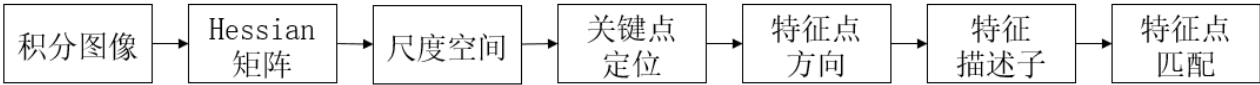


通过插值计算，可能的最小尺度值为 $\sigma = 1.2 \times \frac{(15+9)/2}{9} = 1.6$ 对应的模板尺寸为 12×12 ；可能的最大尺度值为 $\sigma = 1.2 \times \frac{(27+21)/2}{9} = 3.2$ ，对应的模板尺寸为 24×24 。

知乎 @阿耀

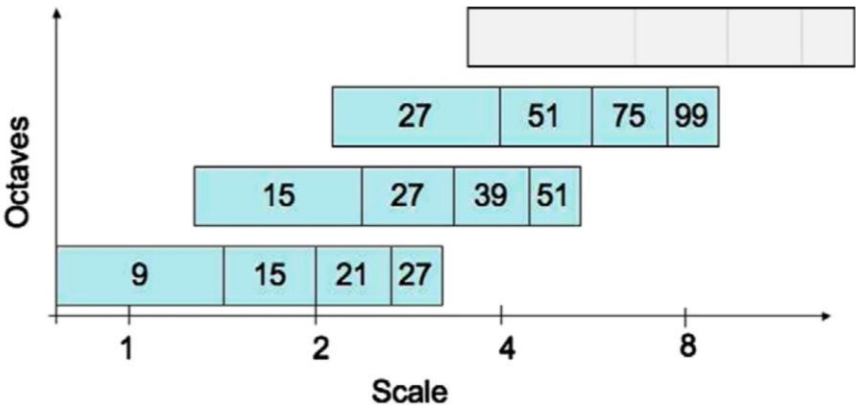


SURF



3.尺度空间表示

采用类似的方法处理其他组的模板序列，其方法是 将滤波器尺寸增量按组数 m 翻倍，即 $6 \times 2^{m-1}$ ，增加数依次为(6,12,24,48,...)，则在盒式滤波金字塔中，每组滤波器的尺寸如下图所示，第二组和第三组的滤波器尺寸分别为(15, 27, 39, 51)和(27, 51, 75, 99)。滤波器的组数可由原始图像的尺寸决定。水平轴代表尺度，组之间有相互重叠，其目的是为了覆盖所有可能的尺度。



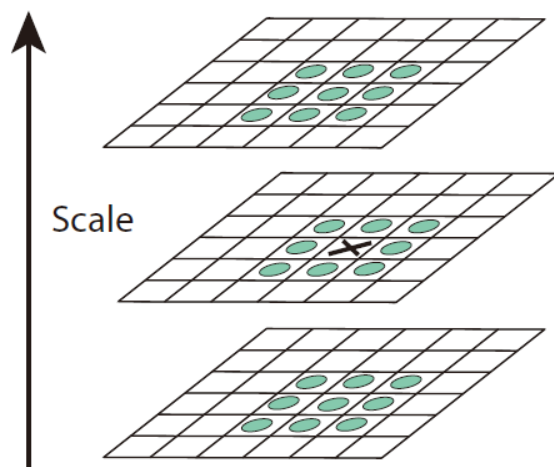
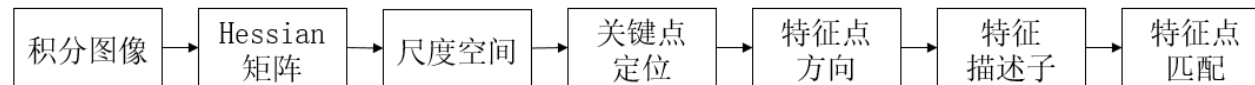
在通常尺度分析情况下，随着尺度的增大，被检测的特征点数迅速衰减。所以一般进行3-4组就可以了，与此同时，为了减少运算量，提高计算的速度，可以考虑在滤波时将采样间隔设为2。

滤波器的尺寸 L 、滤波响应长度 l 、组索引 o 、层索引 s 、尺度 σ 之间的相互关系如下：

$$L = 3 \times (2^{\sigma+1}(s + 1) + 1)$$
$$l = \frac{L}{3} = 2^{\sigma+1}(s + 1) + 1$$
$$\sigma = 1.2 \times \frac{L}{9} = 1.2 \times \frac{l}{3}$$

SURF

4.关键点定位

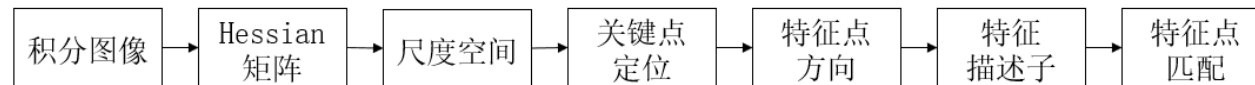


建立尺度空间后，需要搜索定位关键点。将经过盒式滤波处理过的响应图像中每个像素点与其3维邻域中的26个像素点进行比较，若是最极大值点，则认为是该区域的局部特征点。然后，采用3维线性插值法得到亚像素级的特征点，同时去掉一些小于给定阈值的点，使得极值检测出来的特征点更稳健。

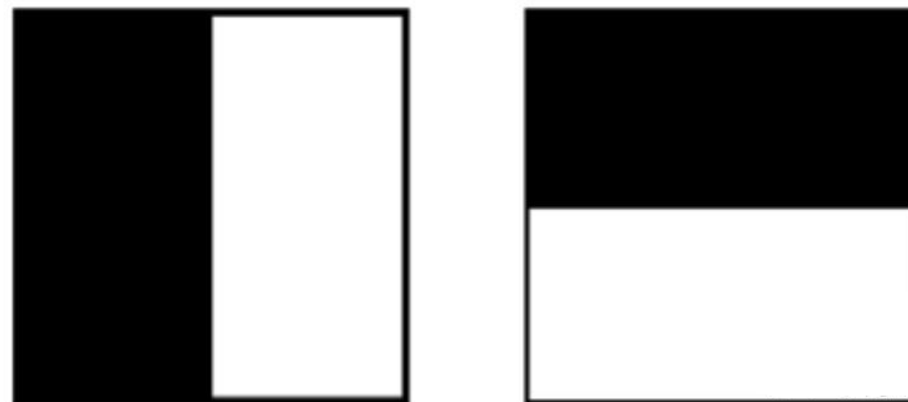
这里和DoG不同的是不用剔除边缘导致的极值点了，因为Hessian矩阵的行列式就已经考虑到边缘的问题了，而DoG计算只是把不同方向变化趋势给出来，后续还需要使用Hessian矩阵的特征值剔除边缘产生的影响。

SURF

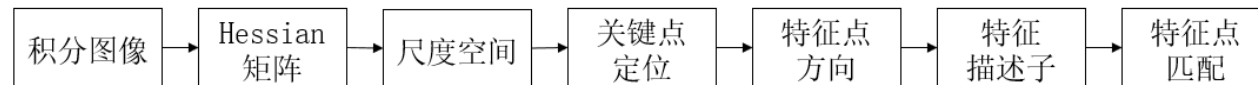
5. 特征点方向匹配



为了保证特征描述子具有旋转不变性，与SIFT一样，需要对每个特征点分配一个主方向。为此，在以特征点为中心，以 $6s$ (s 为特征点的尺度，计算过程为： $s = 1.2 \times L/9$)为半径的区域内，计算图像的 Haar 小波响应，实际上就是对图像进行梯度运算，只不过需要利用积分图，提高梯度计算效率。求Haar小波响应的图像区域和Haar小波模板如右图所示，用于计算梯度的Haar小波的尺度是 $4s$ ，扫描步长为 s 。



SURF



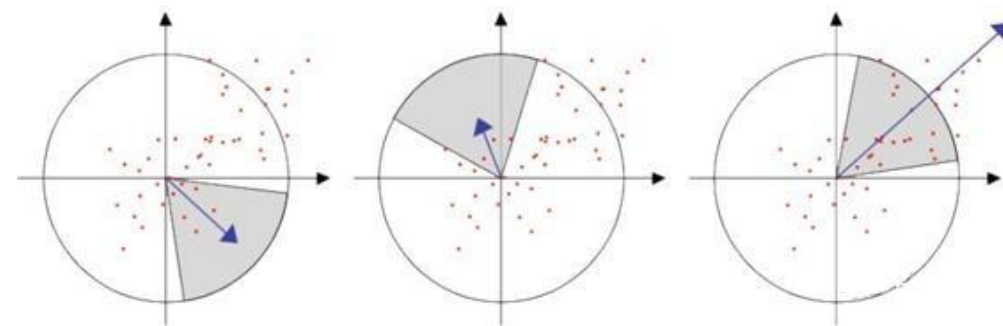
5. 特征点方向匹配

使用 $\sigma = 2s$ 的高斯函数对Haar小波的响应值进行加权。为了求主方向，设计一个以特征点为中心，张角为 $\pi/3$ 的扇形窗口，以一定旋转角度 θ 旋转窗口，并对窗口内的Haar小波响应值 dx, dy 进行累加，得到一个矢量 (m_w, θ_w) ：

$$m_w = \sum_w dx + \sum_w dy$$

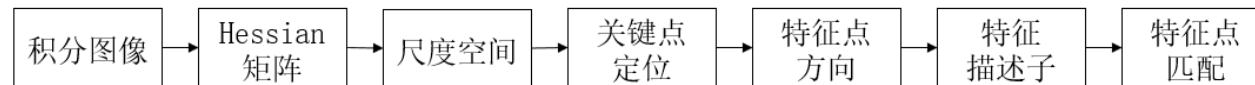
$$\theta = \arctan(\sum_w dy / \sum_w dx)$$

主方向为最大Haar响应累加值所对应的方向，即 $\theta = \theta_w | \max\{m_w\}$ ，当存在大于主峰值80%以上的峰值时，则将对对应方向认为是该特征点的辅方向。一个特征点可能会被指定多个方向，可以增强匹配的鲁棒性。其过程示意图如右。



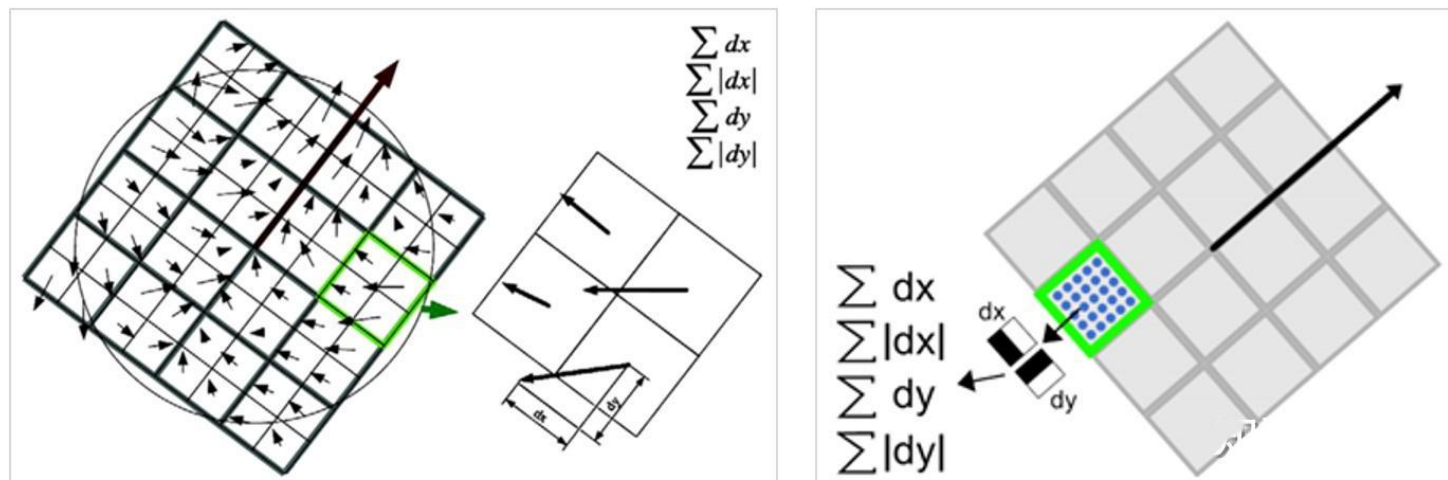
求取主方向时扇形滑动窗口围绕特征点转动，统计Haar小波响应值，并计算方向角

SURF

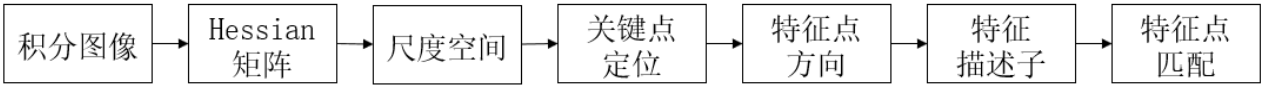


6. 特征描述子生成

生成特征点描述子时，同样需要计算图像的Haar小波响应。与确定主方向不同的是，这里不再使用圆形区域，而是在一个矩形区域计算Haar小波响应。以特征点为中心，沿主方向将 $20s \times 20s$ 的邻域划分为 4×4 个子块，每个子块利用尺寸为 $2s$ 的Haar模板计算响应值，然后对响应值统计水平方向值、水平方向绝对值、垂直方向、垂直方向绝对值已形成特征向量，如下图所示：



SURF



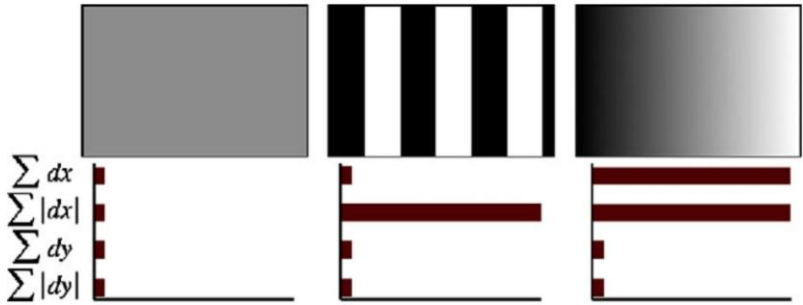
6. 特征描述子生成

首先将 $20s$ 的窗口划分为 4×4 个子窗口，每个子窗口大小为 $5s \times 5s$ ，使用尺寸为 $2s$ 的Haar小波计算子窗口的响应值；然后，以特征点为中心，用 $\sigma 20s/6 = 3.3s$ 的高斯核函数对 dx 、 dy 进行加权计算；最后，分别对每个子块的加权响应值进行统计，得到每个子块的向量：

$$V_i = [\sum dx, \sum |dx|, \sum dy, \sum |dy|]$$

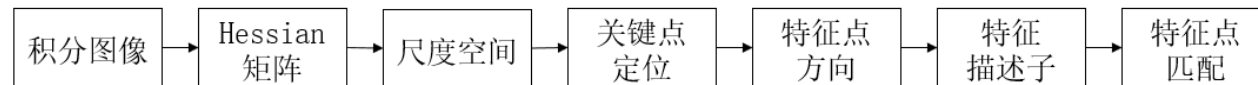
由于共有 4×4 个子块，特征描述子的特征维数为 $4 \times 4 \times 4 = 64$ 。

SURF描述子不仅具有尺度和旋转不变性，还具有光照不变性，这由小波响应本身决定，而对比度不变性则是通过将特征向量归一化来实现。从下图为3种简单模式图像及其对应的特征描述子可以看出，引入Haar小波响应绝对值的统计和是必要的，否则只计算 $\sum dx, \sum dy$ 的话，第一幅图和第二幅图的特征表现形式是一样的，因此，采用4个统计量描述子区域使特征更具有区分度。



图像局部特征点检测

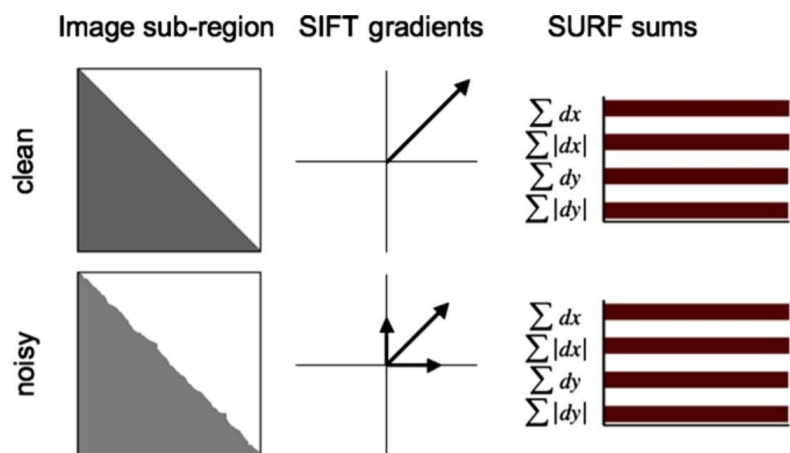
SURF



6. 特征描述子生成

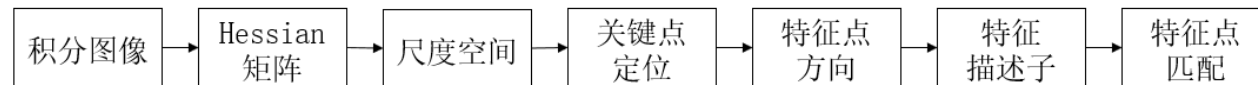
为了充分利用积分图像计算Haar小波的响应值，在具体实现中，并不是直接通过旋转Haar小波模板求其响应值，而是在积分图像上先使用水平和垂直的Haar模板求得响应值 dx 、 dy ，再根据主方向旋转 dx 和 dy 与主方向保持一致。然后对 dx 、 dy 进高斯加权处理，并根据主方向的角度，对 dx 、 dy 进行旋转变换，从而得到旋转后的 dx' 、 dy' 。

SURF在求描述子特征向量时，是对一个子块的梯度信息进行求和，而SIFT是依靠单个像素计算梯度的方向。如下图所示，在有噪声的干扰下，SURF描述子具有更好的鲁棒性。通常特征向量的长度越长，所承载的信息量就越大，特征描述子的独特性就越好，但匹配时的时间代价也越大。



对于SURF描述子，可以将其扩展到128维。具体方法就是在求Haar小波响应值的统计和时，区分 $dx \geq 0$ 和 $dx < 0$ 的情况，以及 $dy \geq 0$ 和 $dy < 0$ 的情况。同时，在求取 $\sum dy$ 、 $\sum |dy|$ 时区分 $dx < 0$ 和 $dx \geq 0$ 情况。这样，每个子块就产生了8个梯度统计值，从而使描述子特征矢量的长度增加到 $8 \times 4 \times 4 = 128$ 维。

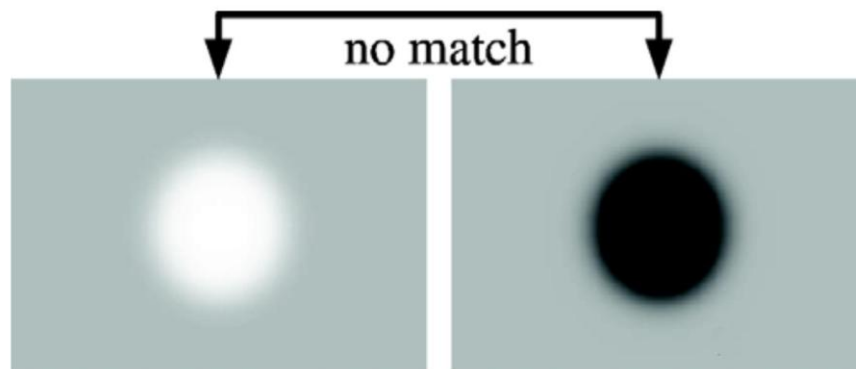
SURF



7. 特征点匹配

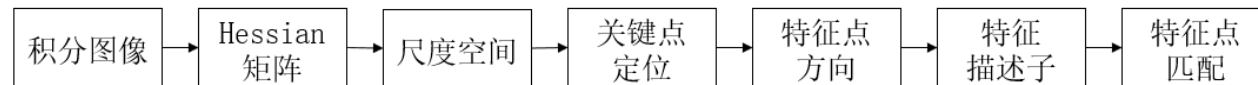
为了实现快速匹配，SURF在特征矢量中增加了新的变量，即特征点的拉普拉斯响应正负号。在特征点检测时，将Hessian矩阵的迹的正负号记录下来，作为特征矢量中的一个变量。这样做并不增加运算量，因为特征点检测时已经对Hessian矩阵的迹进行了计算。**在特征匹配时，这个变量可以有效地节省搜索的时间，因为只有两个具有相同正负号的特征点才有可能匹配，对于正负号不同的特征点就不进行相似性计算。**

简单地说，可以根据特征点的响应值符号，将特征点分成两组，一组是具有拉普拉斯正响应的特征点，一组是具有拉普拉斯负响应的特征点，匹配时，只有符号相同组中的特征点才能进行相互匹配，这样可以节省特征点匹配的时间。如下图所示，黑背景下的亮斑和白背景下的黑斑 因为它们的拉普拉斯响应正负号不同，不会对它们进行匹配。





SURF总结



SURF之所以可以提高速度，主要是SURF对SIFT算法的一些步骤进行近似，达到良好的效果的同时节省了计算资源。

在特征点检测步骤，SIFT用的是高斯模糊和下采样；而SURF用的是通过盒式滤波器对高斯二阶导模板进行近似，同时借助积分图像计算Hessian矩阵，Hessian矩阵的行列式的局部最大值对应为我们所要的兴趣点（特征点）。

在尺度空间建立的步骤，SIFT建立的是高斯金字塔，不断改变原图像的大小，各层之间不改变滤波器的大小；SURF不改变原图像大小，改变的是盒式滤波器的尺度大小，可以并行对图像进行处理，提升了处理速度。

在主方向确定的时候，SIFT采用的是梯度方向直方图，通过直方图找到主方向和可能存在的辅方向；SURF通过以兴趣点为圆心的 $6s$ 圆形区域内的 x 和 y 方向小波响应，在 $\pi/6$ 的区域内进行统计，确定主方向。

在兴趣点描述步骤，SIFT是 $4 \times 4 \times 8 = 128$ 维；而SURF是 $4 \times 4 \times 4 = 64$ 维。



SURF与SIFT对比

尺度空间：SIFT使用DoG金字塔与图像进行卷积操作，而且对图像有做降采样处理；SURF是用近似DoH金字塔(不同尺度的盒式滤波)与图像做卷积，借助积分图，实际操作只涉及到数次简单的加减运算，而且不改变图像大小。

特征点检测：SIFT是先进行非极大值抑制，去除对比度低的点，再通过Hessian矩阵剔除边缘点。而SURF是计算Hessian矩阵的行列式值，再进行非极大值抑制。

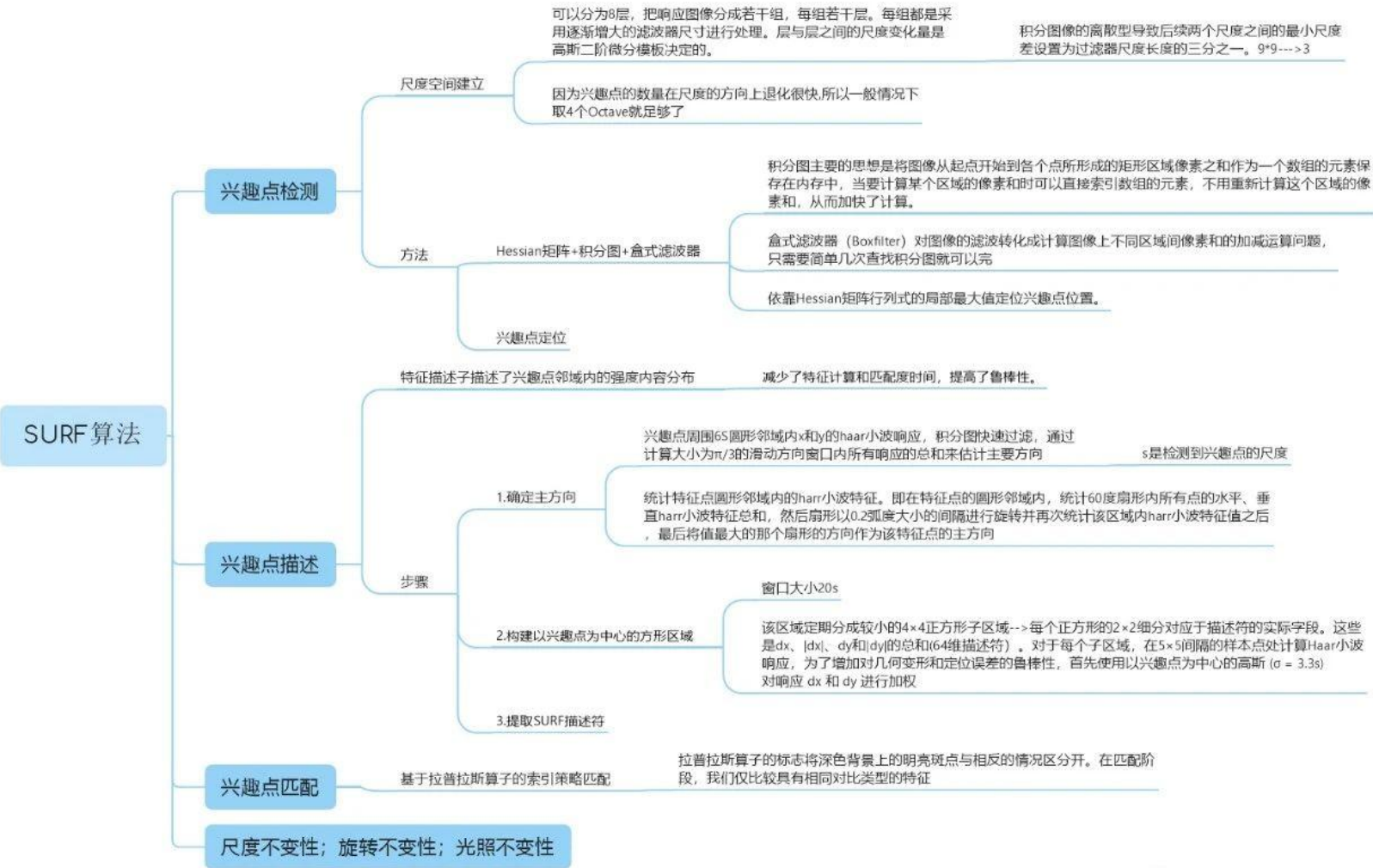
特征点主方向：SIFT在方形邻域窗口内统计梯度方向直方图，并对梯度幅值加权，取最大峰对应的方向；SURF是在圆形区域内，计算各个扇形范围内x、y方向的Haar小波响应值，确定响应累加和值最大的扇形方向。

特征描述子：SIFT将关键点附近的邻域划分为 4×4 的区域，统计每个子区域的梯度方向直方图，连接成一个 $4 \times 4 \times 8 = 128$ 维的特征向量；SURF将 $20s \times 20s$ 的邻域划分为 4×4 个子块，计算每个子块的Haar小波响应，并统计4个特征量，得到 $4 \times 4 \times 4 = 64$ 维的特征向量。

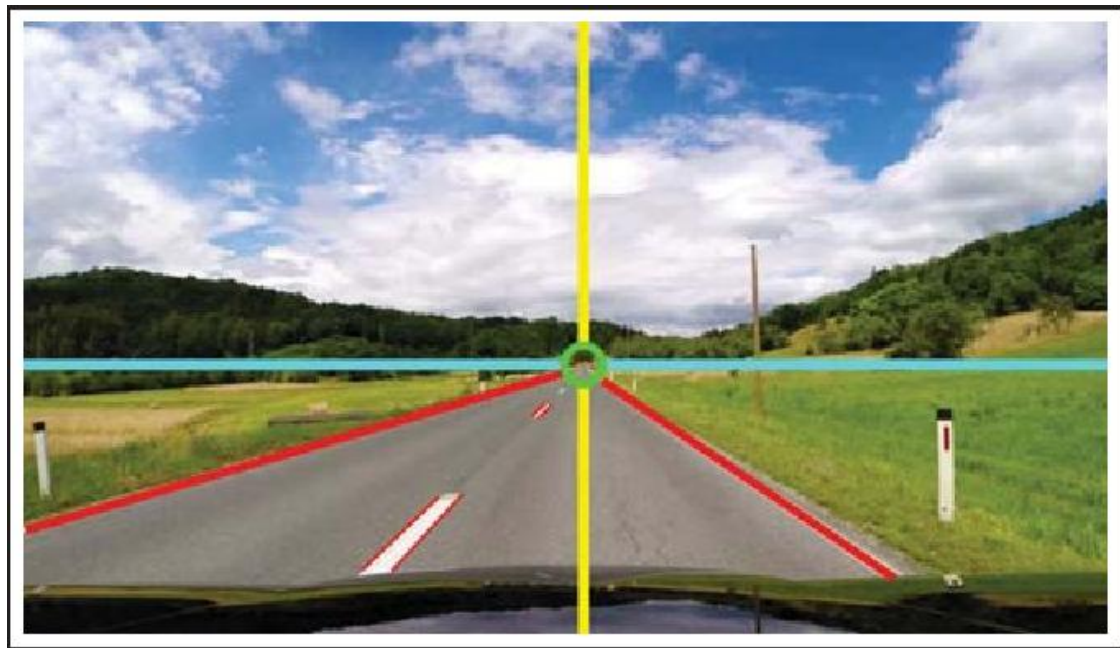


SURF

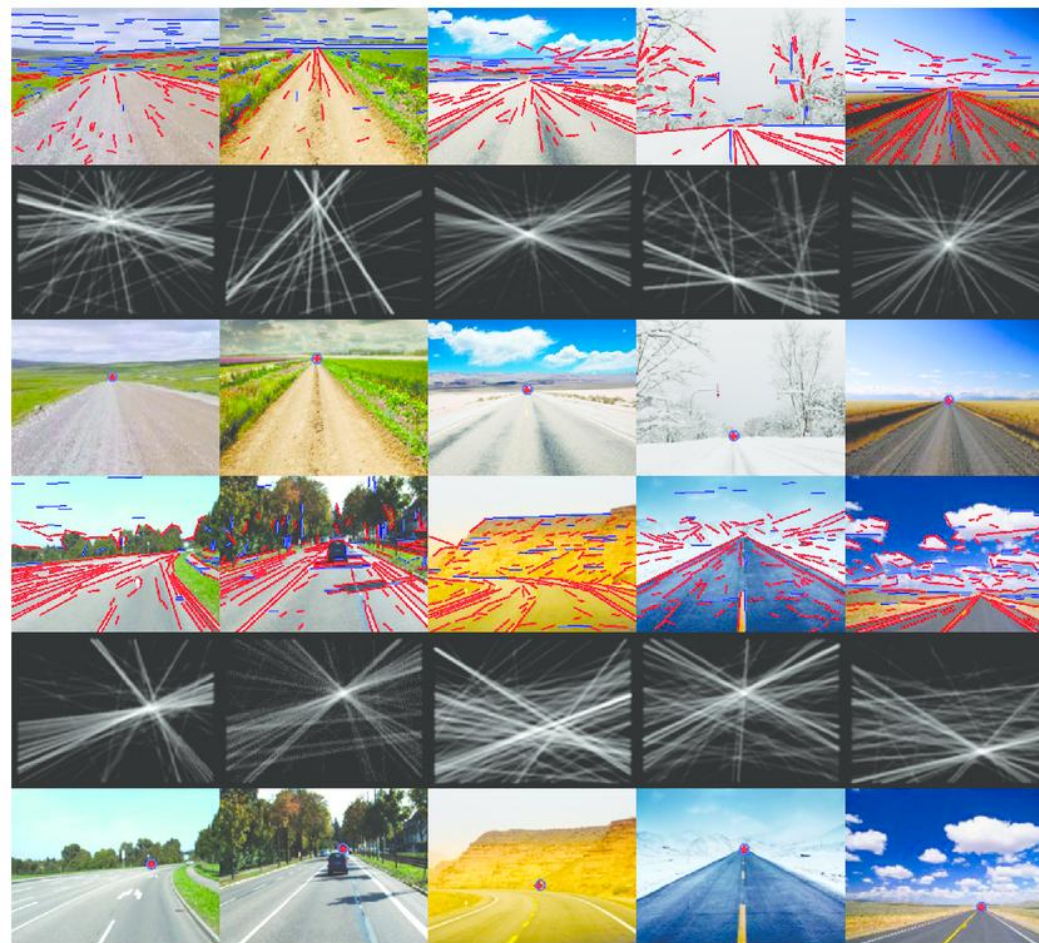
SURF算法总结



消失点检测

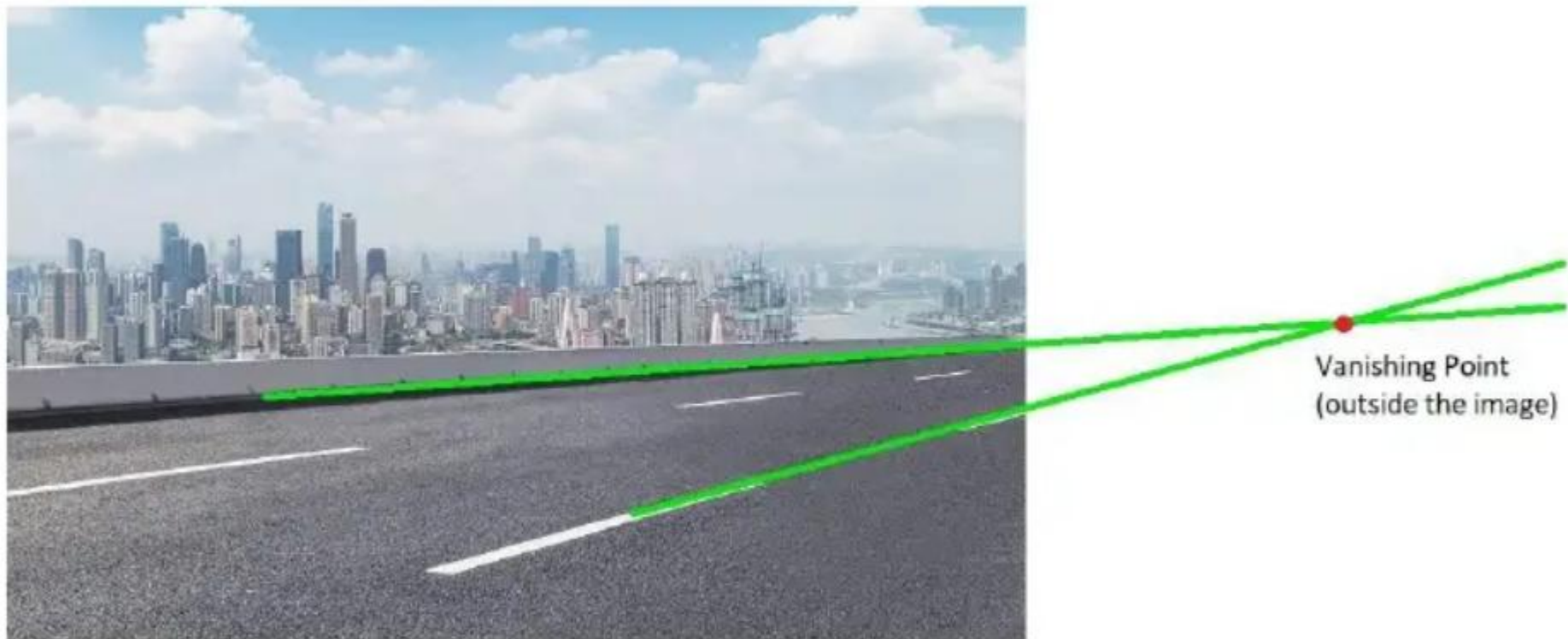


消失点 (Vanishing Point, 又叫灭点)



常见于平行或垂直的建筑物边缘、斑马线、
火车轨道、边缘等

消失点检测



有的消失点已经在图像外



Approach

To solve this problem, we will implement the following easy steps:

- First, we will find all the lines present in the image. These lines should be at least a few pixels long.
- Secondly, we will filter these found lines. Filtering will be done wrt the line's angle wrt the horizontal and their length. The explanation is provided in the respective section.
- Finally, we will find the vanishing point of the image with the help of the lines found in the above 2 steps. It is to note that the vanishing point is approximately the point of intersection of these lines.

消失点检测

直接看Lecture7 - 灭点检测.pdf



利用Canny算子得到边缘图

消失点检测

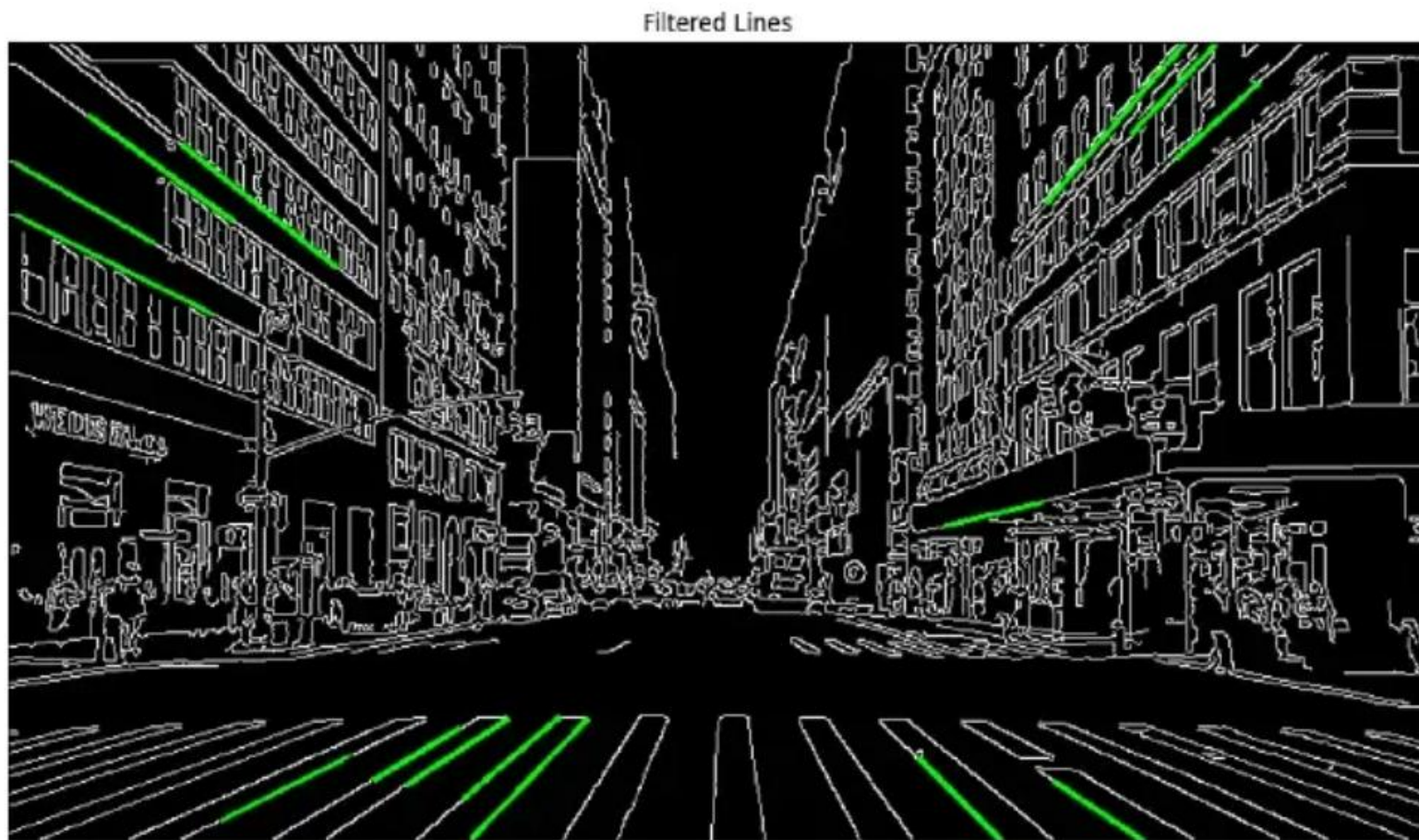
直接看Lecture7 - 灭点检测.pdf



利用霍夫变换找到直线 (HoughLinesP)

消失点检测

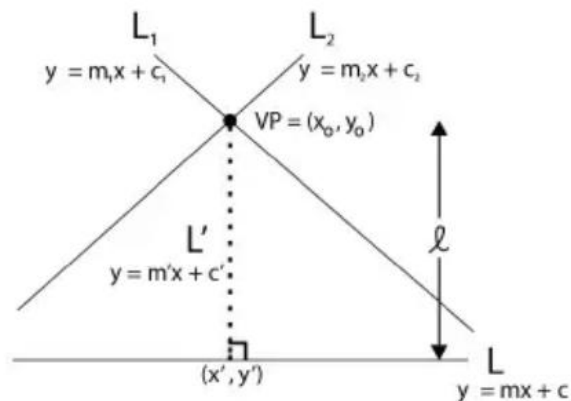
直接看Lecture7 - 灭点检测.pdf



根据角度与长度进行直线过滤

消失点检测

直接看Lecture7 - 灭点检测.pdf



$$l = \sqrt{(y' - y_0)^2 + (x' - x_0)^2}$$

$$\text{Error} = \sqrt{l^2 + \dots}$$

$$x_0 = \left\{ \frac{c_1 - c_2}{m_2 - m_1} \right\}$$

$$y_0 = m_1 x_0 + c_1$$

$$m' = -\frac{1}{m} \quad [L \text{ \& } L' \text{ are perpendicular}]$$

$$c' = y_0 - m' x_0 \quad [\text{As } (x_0, y_0) \text{ is a point on } L']$$

$$x' = \left\{ \frac{c - c'}{m' - m} \right\}$$

$$y' = m' x' + c'$$

Taking lines L_1 and L_2 whose intersection point is (x_0, y_0) .

The equation of line L_1 is $y = m_1 * x + c_1$

The equation of line L_2 is $y = m_2 * x + c_2$

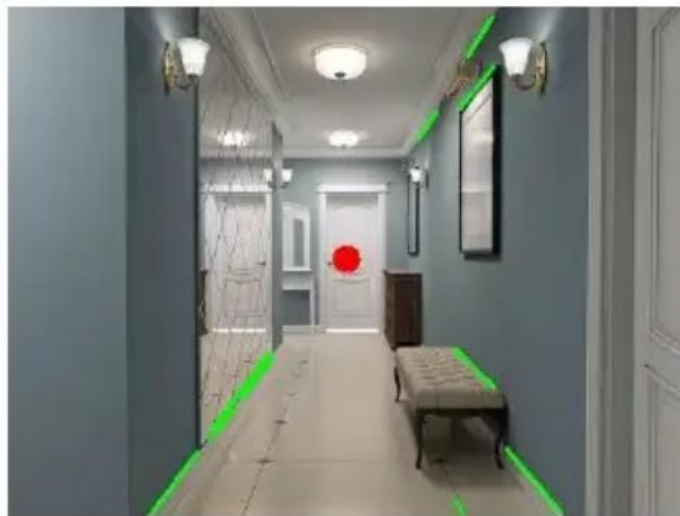
The distance of this point from the line L is $y = m * x + c$

To do so, we make a line L' (L dash) (equation: $y = m' * x + c'$) perpendicular to L and passing through the point (x_0, y_0) .

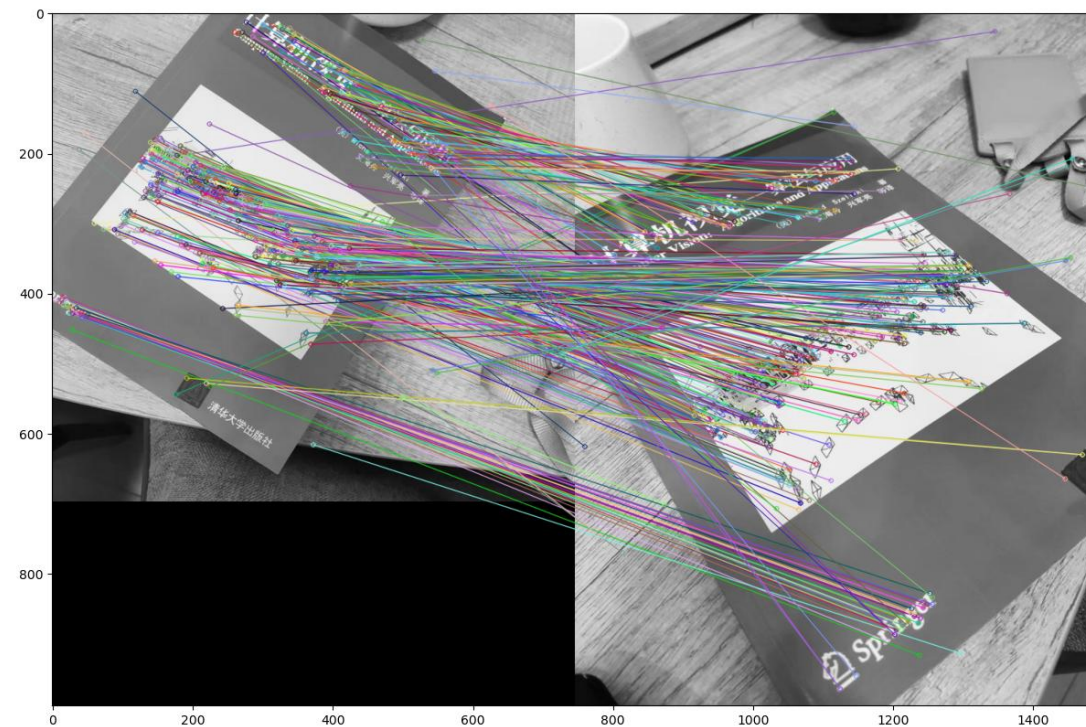
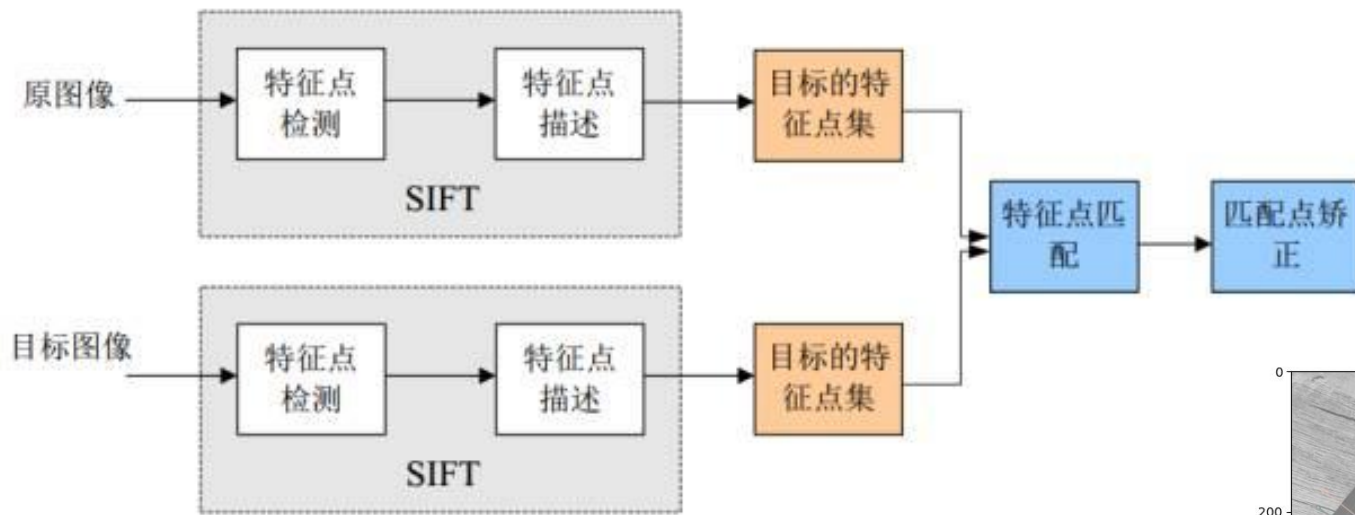
The point of intersection of line L' and L is (x', y') ,
Thus, the distance of the point (x_0, y_0) from line L is distance between (x_0, y_0) and (x', y')

消失点检测

直接看Lecture7 - 灭点检测.pdf
看代码示例: `vanish_point.py`



特征点匹配



特征点匹配

Brute-Force Matching (暴力匹配)

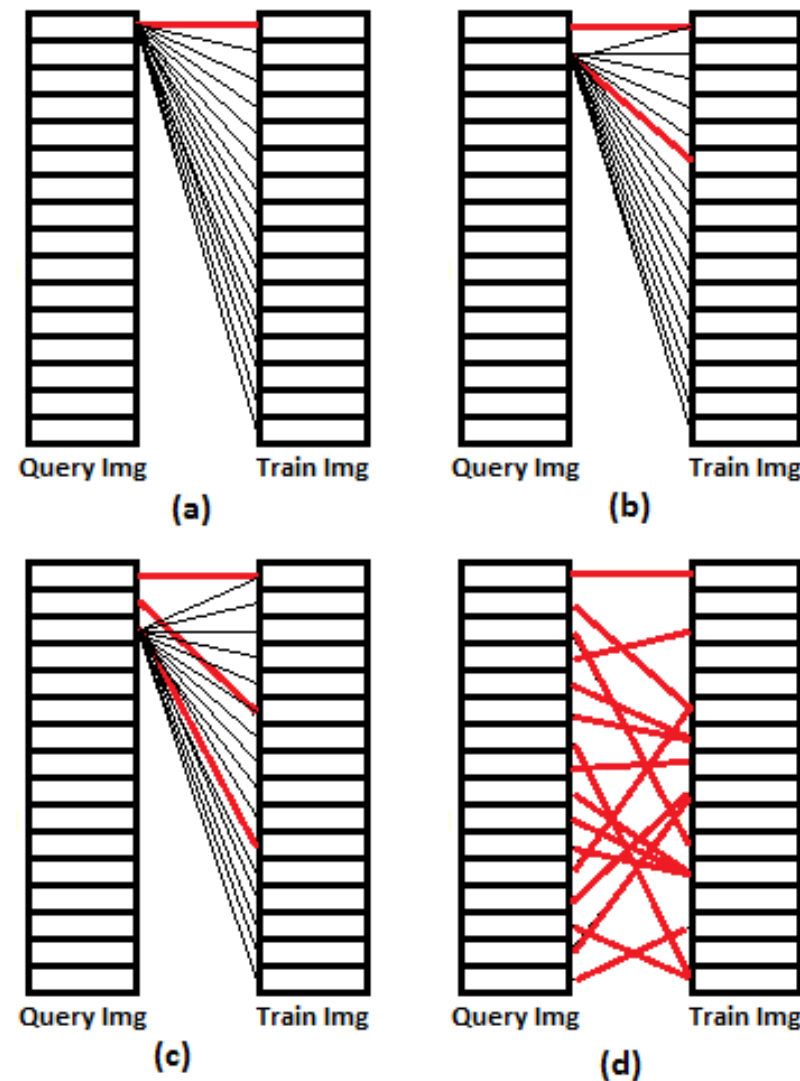
Brute-Force matcher is simple. It takes the descriptor of one feature in first set and is matched with all other features in second set using some distance calculation. And the closest one is returned.

暴力法（也称穷举法、枚举法或蛮力法）是指采用遍历（扫描）技术，即采用一定的策略将待求解问题的所有元素依次处理一次，从而找出问题的解。

算法简单，但执行效率低。因此暴力法的关键在于提高编程效率。

蛮力法不是一个最好的算法（巧妙和高效的算法很少出自蛮力），但当我们想不出更好的办法时，它也是一种有效的解决问题的方法。

它可能是唯一一种几乎什么问题都能解决的一般性方法，是一种非常基本、但又十分重要的算法设计方法



特征点匹配

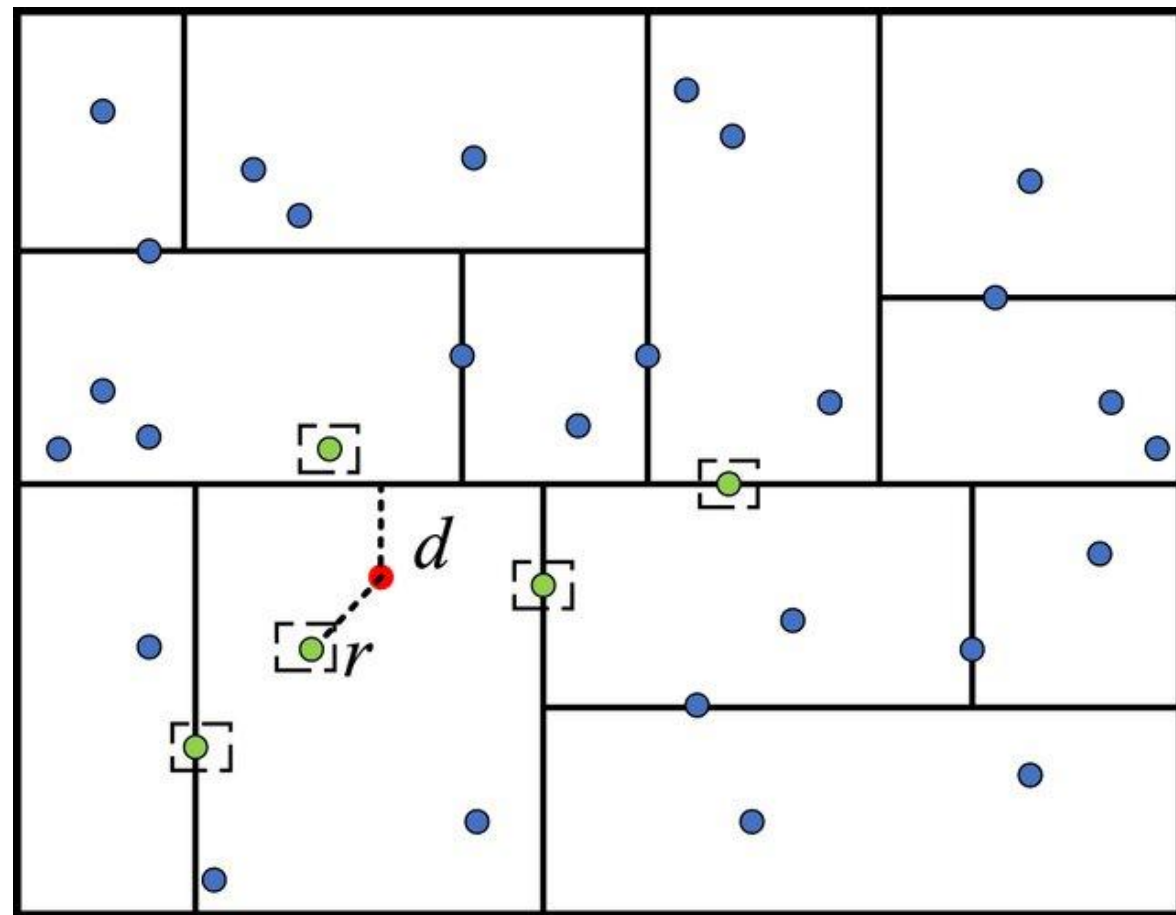
FLANN:

It contains a collection of algorithms optimized for fast nearest neighbor search in large datasets and for high dimensional features. It works faster than BFMatcher for large datasets.

FLANN利用了kdtree或者kmeans来对特征进行聚类建模，查找时可以快速找到最近邻点。具体来说：

FLANN中的算法例如k-d树可以在特征点搜索前根据特征点向量的信息将特征点集合以k-d树的数据结构进行构建，当搜索一个特征点度量距离最近的特征点时，可以由之前构建的数据结构进行搜索，减少搜索的复杂度。所以，FLANN中就包含了多种构建类似k-d树的方法，最常用的还有八叉树等。

具体详解：<https://zhuanlan.zhihu.com/p/617234731>



FLANN searching method with KD tree



Hungarian Algorithm (匈牙利算法) :

1. 先计算Cost Matrix
2. 后面计算他们的最优匹配

优化思路:

1. 从粗到细
2. 根据阈值先去掉部分

```
[5] import numpy as np
    from scipy.optimize import linear_sum_assignment

    cost_matrix = np.array([
        [10, 15, 9],
        [9, 18, 5],
        [6, 14, 3]
    ])

    matches = linear_sum_assignment(cost_matrix)
    print('scipy API result:\n', matches)

    scipy API result:
    (array([0, 1, 2]), array([1, 2, 0]))
```

参考链接: https://mp.weixin.qq.com/s/_COlq6l6OAntYDaM3Nv8OA



Hungarian Algorithm（匈牙利算法）：

	Car D	Car E	Car F
Car A	10	15	9
Car B	9	18	5
Car C	6	14	3

	Car D	Car E	Car F
Car A	1	6	0
Car B	4	13	0
Car C	3	11	0

假设我们现在就有两帧的图片，第一帧我们看到图片中有CarA, CarB, CarC, 并且通过我们的目标检测器找到了这三辆车；第二帧，同样，也看到了三辆车，CarD, CarE, CarF，两帧都只存在这三辆车，但由于目标检测器只能分辨出他们是车，而不能分辨出CarD对应的是CarA还是其它车

第一步：每行减去最小值



特征点匹配

Hungarian Algorithm (匈牙利算法) :

	Car D	Car E	Car F
Car A	1	6	0
Car B	4	13	0
Car C	3	11	0

知乎 @水木皇工仔

第一步：每行减去最小值

	Car D	Car E	Car F
Car A	0	0	0
Car B	3	7	0
Car C	2	5	0

知乎 @水木皇工仔

第二步：每列减去最小值

特征点匹配

Hungarian Algorithm (匈牙利算法) :

	Car D	Car E	Car F
Car A	0	0	0
Car B	3	7	0
Car C	2	5	0

知乎 @水木星工仔

	Car D	Car E	Car F
Car A	0	0	0
Car B	3	7	0
Car C	2	5	0

知乎 @水木星工仔

明显最少的方式应该是两条

第三步：以最少数量的线条划掉所有零，如果这个数量大于等于矩阵的行列数，那么跳到第五步

特征点匹配

Hungarian Algorithm (匈牙利算法) :

	Car D	Car E	Car F
Car A	0	0	0
Car B	3	7	0
Car C	2	5	0

知乎 @水木皇工仔

	Car D	Car E	Car F
Car A	0	0	2
Car B	1	5	0
Car C	0	3	0

知乎 @水木皇工仔

- 剩下的矩阵为 $[[3, 7], [2, 5]]$ ，都减去2
- 然后在交叉的0加上2

第四步：在剩下的矩阵中，减去最小值；如果有零被交叉，那么把这个最小值加上



特征点匹配

Hungarian Algorithm (匈牙利算法) :

	Car D	Car E	Car F
Car A	0	0	2
Car B	1	5	0
Car C	0	3	0

知乎 @水木皇工仔

	Car D	Car E	Car F
Car A	0	0	2
Car B	1	5	0
Car C	0	3	0

知乎 @水木皇工仔

这时我们有3条线了，那么跳到第五步

然后重复第三步

特征点匹配

Hungarian Algorithm (匈牙利算法) :

	Car D	Car E	Car F
Car A	0	0	2
Car B	1	5	0
Car C	0	3	0

知乎 @水木星工仔

	Car D	Car E	Car F
Car A	0	0	2
Car B	1	5	0
Car C	0	3	0

知乎 @水木星工仔

	Car D	Car E	Car F
Car A	0	0	2
Car B	1	5	0
Car C	0	3	0

知乎 @水木星工仔

第二行只有一个零，那么**Car F 对应了car B**，然后**删掉行列**

第三行，**Car C**对应了**Car D**剩下就是**Car A**对应**Car E**了

第五步：从只有一个零的行开始一一对应，对应完则整个行列删除



Hungarian Algorithm (匈牙利算法) :

第一帧	第二帧
Car A	Car E
Car B	Car F
Car C	Car D

知乎 @水木皇工仔

最终结果



特征点匹配

Super Point + Super Glue特征检测和匹配算法

<https://zhuanlan.zhihu.com/p/637191684>



特征点匹配

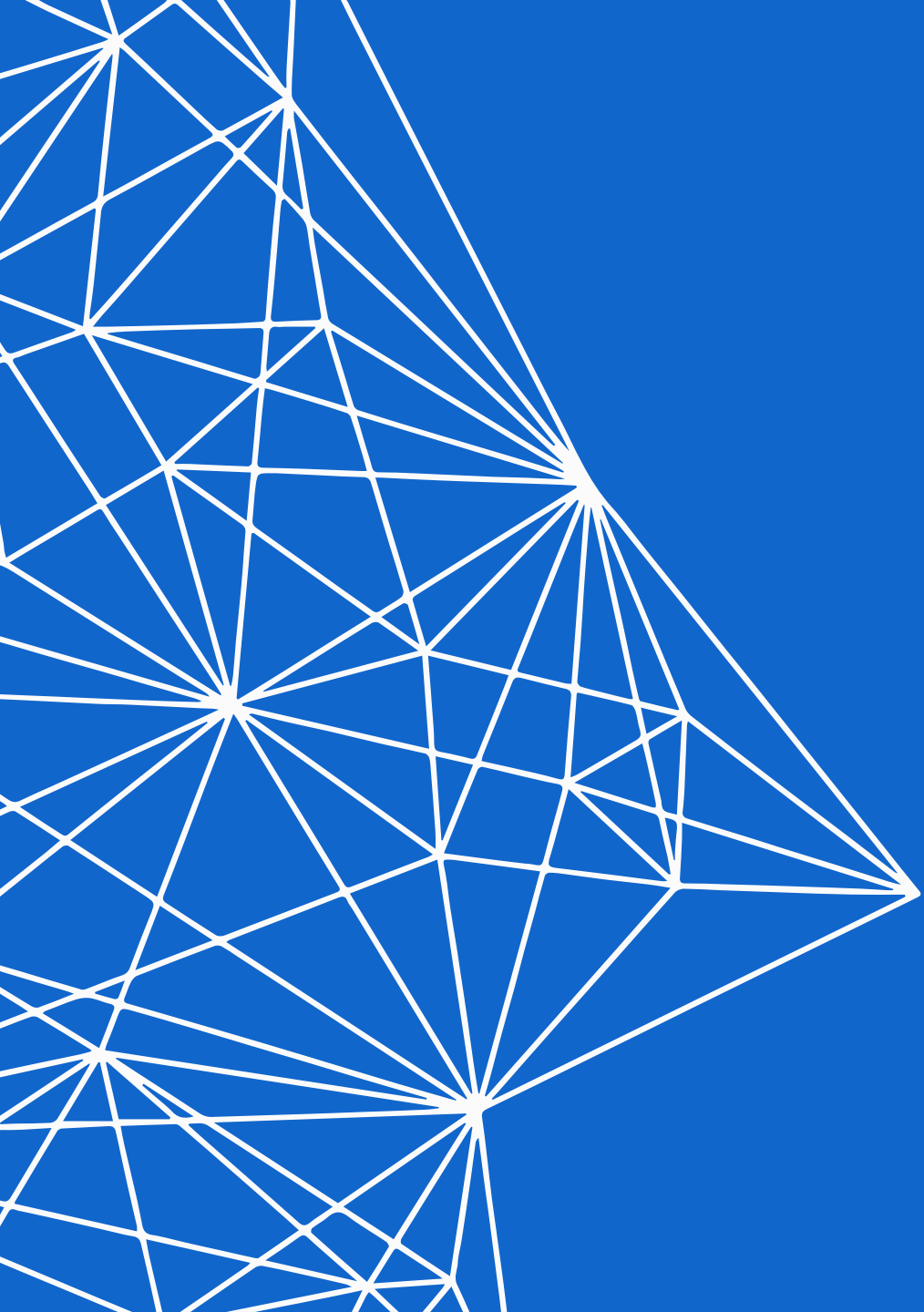
LightGlue

<https://zhuanlan.zhihu.com/p/648869872>



目前火热的Deep Learning会灭绝传统的SIFT/ SURF的特征提取吗？

<https://www.zhihu.com/question/48315686/answer/2562741268>



THANKS!